



QuantumTM Leaps
innovating embedded systems

QSTM **Programmer's Manual**

Document Version 3.2.00
August 2006

Quantum L^eaP^sTM, LLC
www.quantum-leaps.com

Copyright © 2002-2005 **Quantum L^eaP^s, LLC**. All Rights Reserved.

All parts of this document and the referenced software are protected by copyright law and can be used only with a valid license, as described in this document. Any other use or distribution of the referenced software or any part of this document is illegal.

Table of Contents

1	Introduction.....	1
1.1	Software Tracing Concepts.....	2
1.2	Key Features of QS	4
1.3	Licensing QS™	5
1.4	Related Documentation and Resources.....	6
2	Getting Started with QS™	7
2.1	Installation	7
2.2	What You Need to Use QS™	10
2.2.1	Windows or Linux Host QSpy™ Host Application	10
2.2.2	Serial Port and a NULL-Modem Cable	10
2.2.3	Borland Turbo C++ 1.01	10
2.2.4	Linux QS Version	11
2.2.5	Other QS Versions	11
2.2.6	GNU Make	11
2.3	The <stdint.h> Header File	11
2.4	Building the QS Library	12
2.4.1	Building QS Library with Turbo C++ 1.01	13
2.4.2	Building QS Library with gcc for Linux	13
2.5	Building the Examples	14
2.5.1	Building the QHsmTst Example with QS with Turbo C++ 1.01	15
2.5.2	Building the QDPP Example with QS with Turbo C++ 1.01	15
2.5.3	Building The QDPP Example with QS for Linux.....	15
2.6	The QHsmTst Example.....	16
2.6.1	Executing the QHsmTst Example.....	17
2.6.2	Setting-up the QHsmTst Application for Tracing.....	18
2.7	The QDPP-SPY Example for Turbo C++ 1.01	21
2.7.1	Executing the QDPP-SPY Example	21
2.7.2	Initializing QDPP Application for Tracing	26
2.7.3	Defining Platform-Specific QS Callbacks	27
2.7.4	Generating Dictionary Records from Active Objects.....	29
2.7.5	Adding Application-Specific Trace Records	30
2.8	The QDPP-SPY Example for Linux	31
2.8.1	Executing the QDPP-SPY Example	31
2.8.2	Defining Platform-Specific QS Callbacks	32
3	QS Target Component	33
3.1	QS Source Code Structure.....	33
3.2	The QS Platform-Independent Header File qs.h	34
3.3	General Structure of QS Trace Records	36
3.4	Dictionary QS Trace Records	36
3.4.1	Object Dictionaries.....	37
3.4.2	Function Dictionaries	38
3.4.3	Signal Dictionaries	38
3.5	QS Filters	39
3.5.1	Global On/Off Filter	39
3.5.2	Local Filters	40
3.6	Application-Specific QS Trace Records.....	42
3.7	QS Critical Section	43

3.8	QS Data Protocol	44
3.8.1	Transparency	45
3.8.2	Endianness	45
3.9	QS Trace Buffer	46
3.9.1	Byte-Oriented Interface QS_getByte()	46
3.9.2	Block-Oriented Interface QS_getBlock()	46
3.10	Configuring and Porting QS	48
3.10.1	The qs_port.h File	48
3.10.2	The QS Callbacks in the BSP	48
4	QS Compliance with Standards	50
4.1	MISRA™ Compliance	50
4.2	PC-Lint™ Compliance	50
4.3	Coding Standard Compliance	50
4.4	QS Reference Manual	51
5	Related Documents and References	52



1 Introduction

Developers of real-time embedded systems know that getting the code written and compiled is only the first step. The system still needs to be tested, validated, and tuned for best performance and resource consumption. A single-step debugger is frequently not helpful since it stops the system and exactly precludes seeing live interactions within the application. Clogging up high-performance code with `printf` statements is usually too slow and simply unavailable in most embedded systems that don't have screens to print to. So the questions are: How can you monitor the behavior of a running system without degrading the system itself? How do you design and execute repeatable unit and integration tests of your system? How can you discover and document elusive, intermittent bugs that are caused by subtle interactions among concurrent components? How do you ensure that a system runs reliably for long periods of time and gets top processor performance?

Software tracing is a technique for obtaining diagnostic information about running code without the overhead of `printf` statements or use of a debugger. You can apply software tracing to analyze the logic paths, control conditions, processing sequences, data flow, performance, and much more. Software tracing always involves some form of code *instrumentation* to log interesting discrete events for subsequent retrieval from the system and analysis.

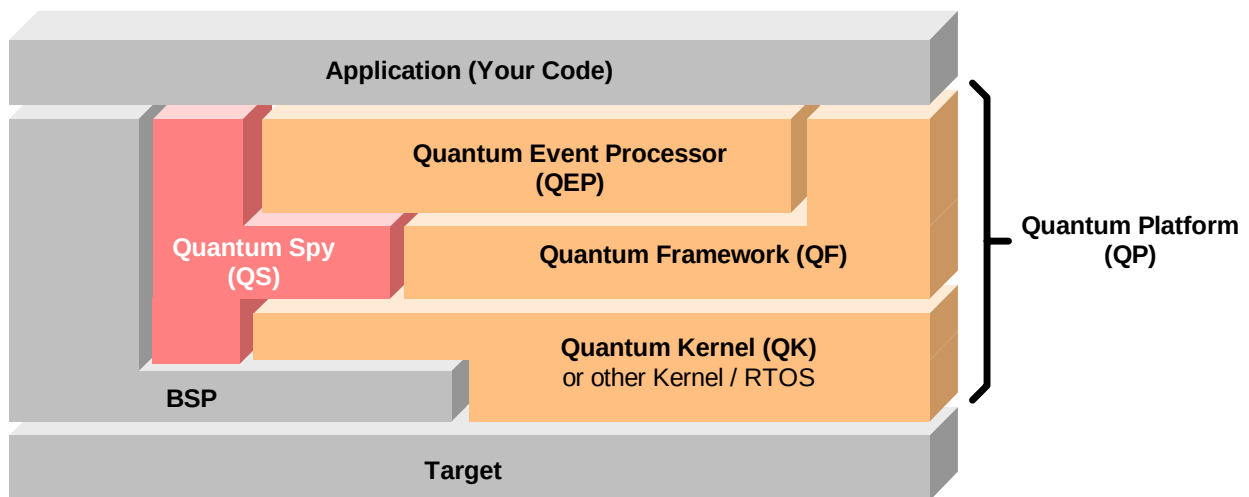


Figure 1 Quantum Spy™ as a component of the Quantum Platform™ (QP).

Quantum Spy™ (QS) is a real-time software tracing facility that is already instrumented in all Quantum Platform (QP) components and is available to the Application code as well (see Figure 1).

Many operating systems provide software tracing capabilities. However, QS takes software tracing to the next level. An instrumented state machine based framework, as opposed to merely an

RTOS, offers unique opportunity for obtaining very comprehensive information about the running system, even without adding any instrumentation to the application code.

As shown in Figure 1, a Quantum Platform application builds on top of the event processor (QEP), the event-driven framework (QF), and the real-time kernel (QK). Therefore, apart from kernel-related events (such as task-state changes, context switch, etc.), the QS trace contains also detailed application-level information, such as state machine activity (state entry, exit, transition), event passing details (queuing, dispatching, recycling), time events (arming, posting, disarming), and much more. QS can provide this wide-ranging tracing data because the framework (QF), the event processor (QEP), and the kernel (QK) completely control the execution of the application, not the other way around as it is the case with traditional operating systems.

NOTE: This Manual covers both the C and C++ version of Quantum Spy (QS). The concrete code examples are taken from the C version, but the C++ version is essentially identical except for some trivial syntax differences.

1.1 Software Tracing Concepts

In the nutshell, software tracing is similar to peppering the code with `printf()` statements for logging and debugging, except that software tracing is much less intrusive and more powerful than the primitive `printf()`. This quick overview introduces the basic concepts and describes some features you can expect from a commercial-grade software tracing system.

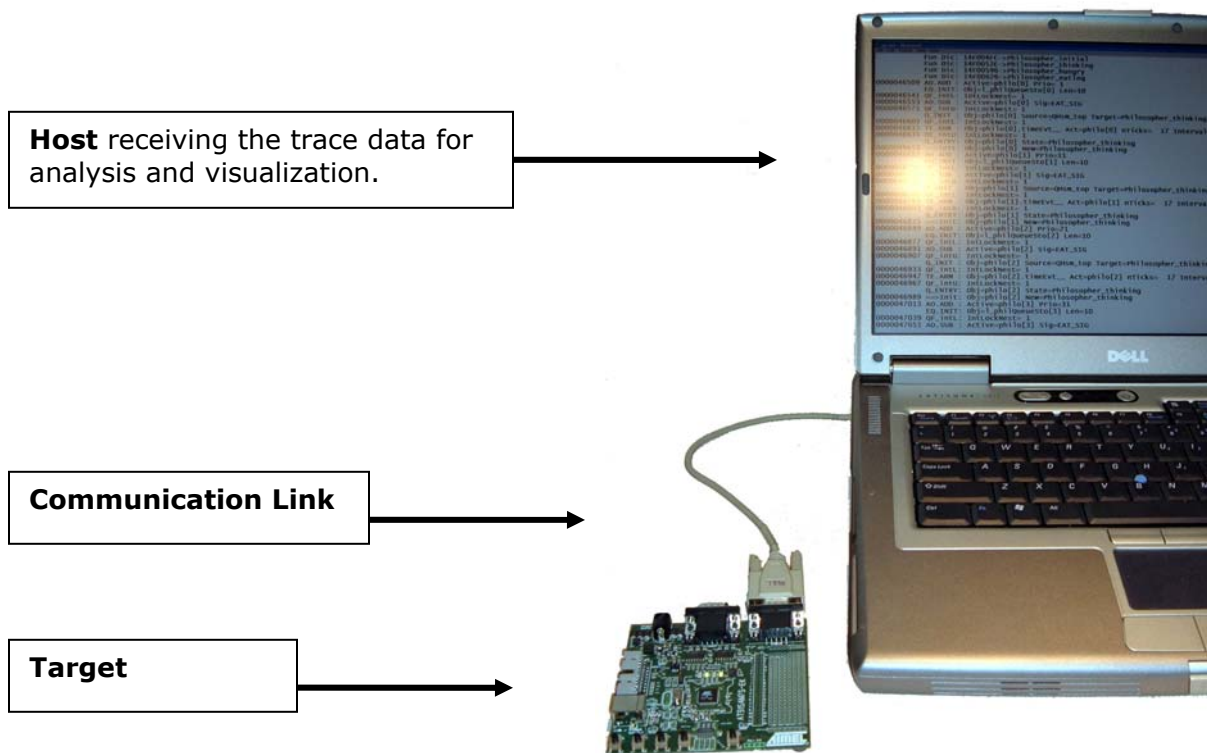


Figure 2 Collecting trace data from the Target to the Host using the Communication Link.

Figure 2 shows a typical setup for software tracing. An embedded Target system is executing instrumented code. The code instrumentation logs the trace data into a RAM buffer inside the Target. From that buffer the trace data is continuously sent over a Data Link to a Host computer, which

stores, displays, and analyzes the information. This configuration means that a software tracing solution consists of two components: a *Target-resident component* for collecting and sending the trace data, and a *Host-resident component* to receive, store, decompress, and analyze the data. Often the host-resident component involves a whole suite of tools for processing and visualizing the data.

A good tracing solution is minimally invasive, which means that it can provide visibility into the running embedded code with minimal impact on the target system behavior. Properly implemented and used, it will let you diagnose a live system without interrupting or significantly altering the behavior of the system under investigation.

Of course, it's always possible that the overhead of software tracing, no matter how small, will have some effect on the target system behavior, which is known as the "*probe effect*" (a.k.a. the "Heisenberg effect"). To help you determine whether that is occurring, you must be able to configure the instrumentation in and out both at run-time as well as at compile-time.

To minimize the "probe effect", a good trace system performs efficient, *selective logging* of trace events using as little processing and memory resources of the target as possible. Selective logging means that the tracing system provides user-definable, fine granularity *filters* so that the target-resident component only collects events of interest — you can filter as many or as few instrumented events as you need. That way you can make the tracing as noninvasive as necessary.

To minimize the RAM usage, the target-resident trace component typically uses a circular trace buffer that is continuously updated, and new data overwrites the old when the buffer "wraps around" due to insufficient transmission rate to the host. This reflects the typically applied "*last-is-best*" philosophy in collecting the trace data. In order to focus on certain periods of time, software trace provides configurable *software triggers* that can start and stop trace collection before the new data overwrites the old data of interest in the circular buffer.

To further maximize the amount of data collected in the trace buffer, the target-resident component typically applies some form of *data compression* to squeeze more trace information into the buffer and to minimize the bandwidth required to uplink the data to the host.

However, perhaps the most important characteristic of a flexible software tracing system is the separation of trace logging ("what information is being traced") from the data transmission mechanism ("where and how is it sent"). This separation of concerns allows the transmissions to occur in the least time-critical paths of the code, such as the idle loop. Also, clients should be able to employ any data transmission mechanism that they can make available on the target. This means both the physical transport layer (e.g., serial port, SPI, USB, Ethernet, etc.) as well as implementation strategy (polling, interrupt, DMA, etc.) The tracing facility should tolerate and be able to detect any RAM buffer overruns due to bursts of tracing data production rate or insufficient transmission rate to the host.

Finally, the tracing facility must allow consolidating data from all parts of the system, including concurrently executing threads and interrupts. This means that the instrumentation facilities must be *reentrant* (i.e., both thread-safe and interrupt-safe).

NOTE: In the embedded systems domain, the days of logic analyzers or in-circuit emulators having direct access to all the CPU's state information are long gone. Even if you had access to all the CPU's address and data signals (which you typically don't, because there are simply not enough pins to go around), the multi-stage pipelines and cache memories make it impossible to figure out what's going on in there. The solution requires testing instrumentation integrated directly into the system's firmware. You can no longer design a system without accounting for testing overhead right from the start. Assuming that all the CPU cycles, the entire RAM, and all the ROM will be devoted strictly to the job at hand simply *won't get the job done*.

1.2 Key Features of QS

Quantum SpyTM (QS) is a software tracing system that allows engineers to perform a live analysis of a running real-time embedded system with minimal target system resources and without stopping or significantly slowing down the code. The key advantages of QS are:

- **Observability, controllability, and testability:** The main advantage of QS compared to other software tracing systems is that it is tightly integrated with all Quantum Platform (QP) components. A running application built of active objects (see the ["QF/C Programmer's Manual"](#)) is a highly structured affair where all important system interactions funnel through the event processor (QEP), the framework (QF), and the kernel (QK). By instrumenting just this tiny "funnel" code you can gain unprecedented insight into the system. The trace data from this part of the system alone, allows you to produce complete sequence diagrams and detailed state machine activity for all state machines in the system. This ability can form the foundation of the whole testing strategy for your embedded application. Also, individual active objects are natural entities for unit testing, which you can perform simply by injecting events into the active objects and collecting the QS trace data. QS makes all this comprehensive execution information available to you even with no instrumentation added to the application-level code.
- **Insight with minimal impact:** QS software tracing is incomparably less intrusive than the primitive `printf()` instrumentation because all the formatting is done after the fact in the host, not in the execution path on the target system. Additionally, the data logging overhead is reduced to storing the data into a RAM buffer, but typically does not include sending the data out of the target device. In QS, data logging and sending to the host are separated so that the target system can typically perform the transmission outside of the time-critical parts of the code, for example in the idle loop of the target CPU.
- **Data compression:** A nice byproduct of avoiding the formatting in the target is a natural data compression. For example, formatted output of a single byte takes two hexadecimal digits (and 3 decimal digits), so avoiding the formatting gives at least a factor of two gain in data density. On top of this natural compression, QS uses such techniques as data dictionaries, and compressed format information, which in practice result in compression factor of 3 to 4 compared to the expanded human-readable format.
- **Selective tracing:** Obviously, QS cannot eliminate completely the overhead of software tracing. But with the fine-granularity filters available in QS, you can make this impact as small as necessary. For greatest flexibility, QS uses two complementary levels of filters (see Figure 9). The first level is filtering based on trace record type, such as entry to a state, or publishing an event. This level works globally for all state machines and event publications in the entire system. The second level of filtering is component-specific. You can setup a filter to trace only a specific state machine object, for example. Combination of such two complementary filtering criteria results in very selective tracing capabilities.
- **Selective triggering:** To minimize the runtime overhead, trace triggering (starting/stopping of the trace) is accomplished in QS with setting up the global trace record-type filters. In particular, you can quickly enable or disable the global filters to start/stop accumulating trace data.
- **High Configurability:** QS has been designed from the ground up to be highly configurable. You can configure all aspects of QS, such as the target trace buffer size, timestamping policy, data transmission implementation and so on.
- **Tight Integration with QP:** QS "knows" about all the object sizes used in QP, such as the `QSignal` size, the event queue counter sizes, the event-pool block size, the time event counter representation and so on, so that only the minimal number of bytes is stored and transmitted

to the host. The QSpy host component is also highly configurable to properly decode the compressed data received from all kinds of targets.

- **Platform-independence:** QS has been designed from the ground up to be platform-neutral. Except of establishing the interrupt locking policy, you don't need to port any QS code to adapt it to your CPU. In particular, the code works the same on little endian and big endian CPUs, and is independent on the machine word size. QS can work with 8-bit, 16-bit, 32-bit, and even 64-bit CPUs.
- **Flexible Timestamping:** Most QS trace records are timestamped. QS provides an efficient API for obtaining platform-specific timestamp information. Given the right timer-counter resource on your target system, you can provide QS with as precise timestamp information as required. The timestamp size is configurable to be 1, 2, or 4 bytes.
- **Block and Character APIs for Transmitting Data:** QS separates data logging from data transmission. QS provides a lightweight API supports various implementation strategies (polling, interrupt, DMA, etc.) and any physical transport layer (e.g., serial port, SPI, USB, Ethernet, etc.)
- **Lightweight but Robust Transmission Protocol:** The data transmission protocol used in QS is one of its greatest strengths. The protocol very is lightweight, but has many the elements of the High Level Data Link Control (HDLC) protocol defined by the International Standards Organization (ISO). The protocol has provisions for detecting transmission errors and allows for instantaneous resynchronization after any error, such as data dropouts due to RAM buffer overruns.
- **Flexible Application-level Tracing:** QS provides facilities to add instrumentation to your application. You can instrument arbitrary trace records into your application code. The QS target component stores them in such a way that the host component always "knows" how to decode the each user trace record.
- **Support for Multiple Target-Host Communication Links:** QS target component and QSpy host application support: file, serial, and TCP/IP communication links "out of the box". Adding other communication links is easy, as will be described in this Manual.
- **Reentrancy:** All QS target instrumentation facilities are reentrant, which means you can use them with multitasking systems and even inside ISRs.
- **Highly-Portable, Customizable Host Component:** The QS host component QSpy is designed to be portable across all popular host platforms (such as Windows and Linux), and contains a Hardware Abstraction Layer (HAL) for implementing input from the various targets. Currently supported mechanisms include serial, UDP/IP, and file.
- **MATLAB® Interface:** The QSpy MATLAB® interface consists of the special MATLAB output file and a MATLAB script to pre-fill MATLAB matrices in the workspace.

1.3 Licensing QSTM

The QSTM component is available only under the royalty-free, commercial licensing. For more information, please refer to the licensing section of our website at <http://www.quantum-leaps.com/licensing/com.htm>.

When you apply for the commercial license, you'll receive the complete source code and free upgrades for one year.

1.4 Related Documentation and Resources

This *Programmer's Manual* describes only the QS target component, but does not cover other related subjects. In particular the **QSpyTM host application**, which is common to both the C and C++ version of QS, is described in the separate document "[QSpy Host Application User's Manual](#)". Other related subjects, such as the active object framework structure, state machines, real-time concepts, and others are described in related documents listed in the Section 5 "References".

2 Getting Started with QS™

This section describes how to install, build, and use QS based on two example applications. This information is intentionally included early in this document, so that you could start experiment with QS as soon as possible.

This *Programmer's Manual* describes the standard QS distribution, which can be augmented with other Quantum Platform components and ports to specific CPUs, boards, and development environments. These additional components assume that the standard QS distribution has been already installed.

NOTE: Please refer to the document [Quantum Leaps Licensing and Support](#) available in PDF or to the HTML page at www.quantum-leaps.com/licensing for details about the terms of licensing and commercial support.

2.1 Installation

QS is available for an unrestricted download from www.quantum-leaps.com/downloads.

NOTE: The code of QS is organized according to the Application Note "[QP Directory Structure](#)".

QS requires at least the compatible Quantum Event Processor™ (QEP) component to be already installed. The more advanced QS example assumes additionally that the compatible QF and QK components have been installed as well.

NOTE: QP components are *compatible* when they belong to the same programming language (C or C++) and have the same major and minor version numbers, but perhaps different release numbers. For example, QS/C 3.1.00 is compatible with QF/C 3.1.01. However, QF/C 2.6.00 is not compatible with QS/C 3.1.04.

The QS/C code is distributed in a ZIP archive `qsc_<ver>.zip` (QS/C++ in archive `qscpp_<ver>`), where `<ver>` stands for a specific QS/C version, such as 3.2.00. You must uncompress the archive into **the same** QP Root Directory in which you installed the compatible QEP and potentially other QP components. The QP Root Directory will be referred henceforth as `<qpc_3>`. The following Listing 1 shows the contents of the QP Root Directory after the installation of both QEP and QS components. Directories and selected files included in the QS distribution are shown in boldface.

<code><qpc_3>/</code>	- QP -Root Directory
<code>+-gpl.txt</code>	- GNU General Public License
<code> </code>	
<code>+-include/</code>	- QP platform-independent include files
<code> +-qep.h</code>	- QEP platform-independent public include file
<code> +-qf.h</code>	- QF platform-independent public include file
<code> +-qk.h</code>	- QK platform-independent public include file
<code> +-qs.h</code>	- QS platform-independent public include file
<code> </code>	
<code>+-examples/</code>	- QP examples
<code> +-80x86/</code>	- QP examples for 80x86 CPU architecture
<code> +-dos/</code>	- DOS port
<code> +-tccpp101/</code>	- Turbo C++ 1.01 compiler
<code> +-l/</code>	- Large memory model

```

+-qhsm-spy/ - QHsmTst application with Spy
+-spy/ - QEP library for Spy build
+-Makefile - make file to build qhsmtst example for the Small model
+-qhsmtst.h - application header file
+-qhsmtst.c - QHsmTst state machine
+-main.c - main entry point
+-include.rsp - response file for the Turbo C++ compiler
+-link_spy.rsp - response file for the Turbo C++ linker (Spy build)
+-qk/ - QK port
+-tcpp101/ - Turbo C++ 1.01 compiler
+-l/ - Large memory model
+-qdpp-spy/ - Quantum Dining Philosopher Problem with Spy example
+-spy/ - Object files, map file, and the executable for Spy build
+-Makefile - make file to build qdpp example for the Large model
+-qdpp.h - application header file
+-bsp.c - Board Support Package
+-main.c - main entry point
+-philo.c - Philosopher active object
+-table.c - Table active object
+-include.rsp - response file for the Turbo C++ compiler
+-link_spy.rsp - response file for the Turbo C++ linker (Spy build)
+-QDPP.PRJ - Turbo C++ 1.01 project file to build and debug qdpp example
+-linux/ - Linux port
+-gcc/ - gcc compiler
+-qdpp-spy/ - Quantum Dining Philosopher Problem with Spy example
+-spy/ - Object files, map file, and the executable for Spy build
+-Makefile - make file to build qdpp example for the Large model
+-qdpp.h - application header file
+-bsp.h - Board Support Package header file
+-bsp.c - Board Support Package
+-main.c - main entry point
+-philo.c - Philosopher active object
+-table.c - Table active object
+-ports/ - QP ports
+-80x86/ - QP ports to 80x86 CPU architecture
+-dos/ - DOS port
+-tcpp101/ - Turbo C++ 1.01 compiler
+-l/ - Large memory model
+-spy/ - QP libraries for Spy build
+-qep.lib - QEP library for Spy build
+-qf.lib - QF library for Spy build
+-qk.lib - QK library for Spy build
+-qs.lib - QS library for Spy build
+-qep_port.h - QEP platform-specific include file
+-qf_port.h - QF platform-specific include file
+-qk_port.h - QK platform-specific include file
+-qs_port.h - QS platform-specific include file
+-qk/ - QK port
+-tcpp101/ - Turbo C++ 1.01 compiler
+-l/ - Large memory model
+-spy/ - QP libraries for Spy build
+-qep.lib - QEP library for Spy build
+-qf.lib - QF library for Spy build
+-qk.lib - QK library for Spy build
+-qs.lib - QS library for Spy build
+-qep_port.h - QEP platform-specific include file
+-qf_port.h - QF platform-specific include file
+-qk_port.h - QK platform-specific include file
+-qs_port.h - QS platform-specific include file
+-linux/ - Linux port
+-gcc/ - gcc compiler
+-spy/ - QP libraries for Spy build
+-qep_port.h - QEP platform-specific include file
  
```

```

| | | +-qf_port.h - QF platform-specific include file
| | | +-qs_port.h - QS platform-specific include file
| | | . . .
| +-lint/ - QP port to lint with a generic C compiler
| | +-qep_port.h - QEP platform-specific include file
| | +-qf_port.h - QF platform-specific include file
| | +-qk_port.h - QK platform-specific include file
| | +-qs_port.h - QS platform-specific include file
|
| +-qep/ - QEP directory
| | +-80x86/ - QEP ports to 80x86 CPU architecture
| | | +-qk/ - QEP port for QK
| | | | +-tcpp101/ - Turbo C++ 1.01 compiler
| | | | | +-l/ - Large memory model
| | | | | +-spy/ - Object files for the Spy build
| | | | | +-Makefile - make file to build QEP library
| | | | . . .
|
| +-qf/ - QF directory
| | +-80x86/ - QF ports to 80x86 CPU architecture
| | | +-qk/ - QF port to QK
| | | | +-tcpp101/ - Turbo C++ 1.01 compiler
| | | | | +-l/ - Large memory model
| | | | | +-spy/ - Object files for the Spy build
| | | | | +-Makefile - make file to build QF library
| | | | . . .
|
| +-qk/ - QK directory
| | +-80x86/ - QK ports to 80x86 CPU architecture
| | | +-qk/ - QK port to QK
| | | | +-tcpp101/ - Turbo C++ 1.01 compiler
| | | | | +-l/ - Large memory model
| | | | | +-spy/ - Object files for the Spy build
| | | | | +-Makefile - make file to build QK library
| | | | . . .
|
| +-qs/ - QS directory
| | +-80x86/ - 80x86 CPU architecture
| | | +-dos/ - QS port to DOS
| | | | +-tcpp101/ - Turbo C++ 1.01 compiler
| | | | | +-l/ - Large memory model
| | | | | +-spy/ - Object files for the Spy build
| | | | | +-Makefile - make file to build QS library for the Large model
| | | | +-qk/ - QS port to QK
| | | | | +-tcpp101/ - Turbo C++ 1.01 compiler
| | | | | | +-l/ - Large memory model
| | | | | | +-spy/ - Object files for the Spy build
| | | | | | +-Makefile - make file to build QS library for the Large model
| | | | . . .
| | +-source/ - QS platform-independent implementation files
| | +-lint/ - QS "port" to PC-lint
| | | +-lin.bat - A batch file to invoke PC-lintTM
| | | +-std.lnt - An indirect PC-lint file with compiler-specific options
| | | +-options.lnt - An indirect PC-lint file with compiler-independent options

```

Listing 1 Selected directories and files after installing QS. The elements in boldface are included in the standard QS distribution. NOTE: The evaluation version of QS does not contain the <qpc_3>/qs/source/ directory.

2.2 What You Need to Use QSTM

The QS source code is written in portable ANSI C (the C++ version uses the portable Embedded C++ subset), so you need an ANSI C (EC++) compiler or a cross-compiler for your platform. You also need appropriate other tools, such as a librarian, linker/locator, and some way of debugging your application.

Your C compiler must provide a mechanism to disable and enable interrupts from C. Some compilers allow you to insert in-line assembly language statements into your C source code, which makes it easy to insert the proper processor instructions to enable and disable interrupts. Other compilers actually contain language extensions to enable and disable interrupts. Alternatively, interrupt enabling and disabling can be done inside C-callable assembler functions.

The QS target component is deployed as a fine-granularity object library for "self configuration", which is automatic elimination of unused features at link-time as opposed to hand-configuration at compile-time via preprocessor macros. Therefore, you'll need a librarian utility to generate the QS object library.

2.2.1 Windows or Linux Host QSpyTM Host Application

As shown in Figure 2, you need the QSpyTM host application to receive the trace data, decompress it, render in a human-readable format, and optionally export the data to MATLAB®. The QSpyTM host application is distributed separately and described in the separate manual "[QSpy Host Application User's Manual](#)".

2.2.2 Serial Port and a NULL-Modem Cable

One of the examples shipped in the standard QS distribution sends the trace data over a serial port of a Windows PC. If your PC has two serial ports, you can produce and collect the data on the same machine by using a NULL-modem cable to connect the producer serial port (attached to the QS-Target component) with the consumer serial port (attached to the QSpy host application). Alternatively, you can use two machines: a Windows PC running the QS example application and a Windows or Linux host to collect the trace data. Again, you need to connect the serial ports of these two machines with a NULL-modem cable (see also Figure 2).

2.2.3 Borland Turbo C++ 1.01

The example applications provided in the standard QS distribution assume the legacy Turbo C++ 1.01 compiler, which is available for a *free* download from the Borland "Museum" at <http://bdn.borland.com/article/0,1410,21751,00.html>.

NOTE: The legacy DOS platform has been chosen for demonstrating QS because it still allows to disable interrupts and directly access I/O space of the processor. No other desktop 32-bit development environment for the commodity PC allows this much so easily. The ubiquitous PC running under DOS (or a DOS console within any variant of Windows) is capable of demonstrating most key embedded features of QS.

To install Borland Turbo C++ 1.01, unzip the TCPP101.ZIP archive into a temporary directory. Run the INSTALL.EXE program and follow the installation instructions. For compatibility with the provided examples, you should install the tool into the directory **C:\tools\tcpp101**. If you choose a different directory, you'll need to modify the make files and the linker response files provided in the standard QS distribution.

2.2.4 Linux QS Version

The standard QS distribution contains the QS version for Linux as a target (as opposed as Linux host running the QSpy host application). The Linux QS example sends the trace data over a TCP/IP link to the host, which can be Linux or Windows.

2.2.5 Other QS Versions

Other QS ports are available for a free download from www.quantum-leaps.com/downloads/-ports.htm. Please keep checking this website frequently because new ports are added and updated often.

2.2.6 GNU Make

To achieve commonality among different platforms, Quantum Leaps software does not use the vendor-specific IDEs, or non-compatible make utilities, such as the Borland make distributed with Turbo C++ 1.01, for building the QP libraries and applications. Instead, QP supports command-line build process conforming to the **GNU make standard**, which in turn conforms to Section 6.2 of IEEE Standard 1003.2-1992 (POSIX.2).

For your convenience, the standard QS distribution contains make files for building the QS libraries and the examples. To take advantage of these make files, you need a **GNU-compatible** make utility. For the Windows host, the MinGW GNU-make is available for a free download from www.mingw.org/download.shtml and also mirrored at www.quantum-leaps.com/downloads/-tools.htm#GNUMake. After installing the utility, you should add the GNU-make directory to your PATH.

NOTE: The Makefiles provided in the QS distribution will only work with a GNU-compatible make utility and will **not** work with the DOS make utility that ships with Turbo C++ 1.01. You should be able to easily modify the provided Makefiles for different CPUs and compilers.

2.3 The <stdint.h> Header File

All QP components, including QS, use the C99 standard file <stdint.h> for portable exact-width integer types (see www.open-std.org/jtc1/sc22/open/n2794 Section 7.18).

The newer C/C++ compilers should support this header file as part of the standard C library. In case this standard header file is not available (e.g., in a pre-standard compiler), you should create it and place in the compiler's include directory. At a minimum, this file should contain the typedefs for the following exact-width integer data types (C99 Standard Section 7.18.1.1) that correspond to the CPU and compiler settings you're using to build QEP and QF.

For QP ports to pre-C99 compilers, a minimal <stdint.h> header file is provided as part of the QEP port. However, because the port cannot anticipate the installation directory of the compiler, you need to manually move the provided stdint.h header file from the port directory to the compiler's header file directory (see Application Note "[QP Directory Structure](#)").

NOTE: Borland Turbo C++ 1.01 is an example of a pre-C99-standard compiler. The QEP distribution includes a minimal stdint.h header file in the <qpc_3>\ports\... directory, which you should move into the INCLUDE directory of your Turbo C++ 1.01 installation.

2.4 Building the QS Library

The standard QS distribution includes a version of the QS library for 80x86/DOS large memory model and a version for Linux.

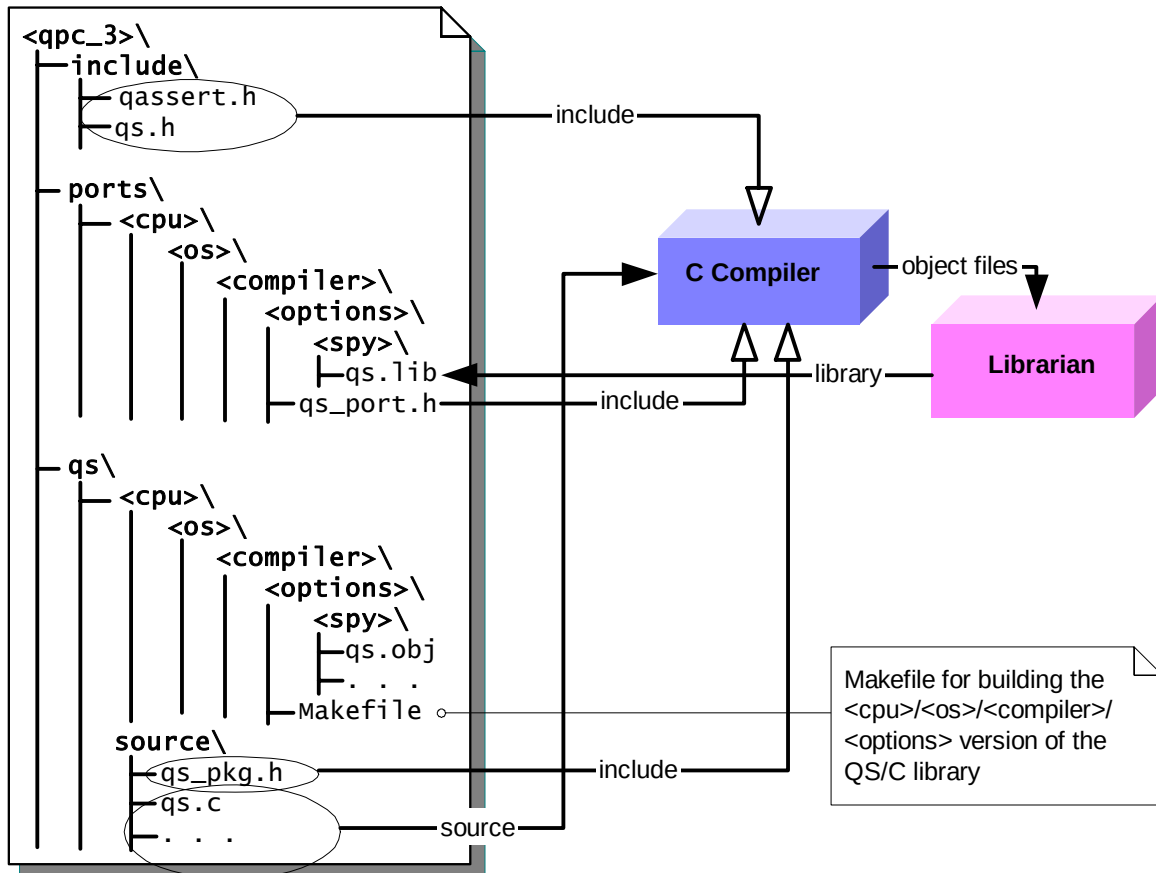


Figure 3 QS library build process

Figure 3 illustrates the general steps required to build a QS library for a given combination of CPU, operating system, compiler, and compiler options. Please note that the C compiler includes the header files from three directories (the include directories are typically specified with the `-I` compiler option): the `<qpc_3>\include\` directory, the QP port directory `<qpc_3>\ports\<cpu>\<os>\<compiler>\<options>\`, and QS source directory `<qpc_3>\qs\source\`. The compiler produces object files in the QS build directory `<qpc_3>\qs\<cpu>\<os>\<compiler>\<options>\<build>\`, where the `<build>\` subdirectory indicates the type of the build (debug, release, etc.).

The librarian takes the object files and produces the QS library in the QP port directory `<qpc_3>\port\<cpu>\<os>\<compiler>\<options>\<build>\`.

All Makefiles provided in the standard QS distribution (such as the one shown in Figure 3) support building the following three different versions of libraries and applications: (1) release version, (2) debug version, and (3) **spy version** for debugging with the Quantum Spy™. The object files and libraries for each version are placed in the separate directories: `dbg`, `re1`, and `spy`, respectively.

You choose the build version by providing a target to the make utility. The default target is "spy". Other targets are "rel", and "dbg" respectively. The following table summarizes the targets accepted by the Makefiles.

Software Version	Build command	Clean command
Debug (default)	make	make clean
Release	make rel	make rel_clean
Spy	make spy	make spy_clean

Table 1 Make targets for the debug, release, and spy software versions

2.4.1 Building QS Library with Turbo C++ 1.01

To make the QS library for 80x86 CPU, DOS, with Turbo C++ 1.01, open a console window on a Windows PC, change directory to <qpc_3>\qs\80x86\qk\tcpp101\1\, and invoke the GNU-make utility by typing at the command prompt the following command:

```
mingw32-make spy
```

The make utility processes the make file Makefile in the current directory to build the QS library. This Makefile assumes that Borland Turbo C++ 1.01 has been installed in the directory C:\tools\tcpp101. You need to adjust the Makefile if you've installed Turbo C++ 1.01 into a different directory. The make process should produce the QF library in the location: <qpc_3>\ports-\80x86\qs\tcpp101\1\dbg\qs.lib.

NOTE: the QS library for Turbo C++ 1.01 as well as the example applications are built with the full symbolic debugging information, which significantly increases the size of these files. Therefore, you should *not* judge the QK memory footprint by the size of the provided object files, or by the size of the QS library.

2.4.2 Building QS Library with gcc for Linux

To build the QS library for Linux with gcc you open a terminal window on a Linux machine, change directory to <qpc_3>/qs/80x86/linux/gcc/, and invoke the GNU-make utility by typing at the command prompt the following command:

```
make spy
```

The Makefile located in the directory <qpc_3>/qs/80x86/linux/gcc/ should produce the QS library in the location: <qpc_3>/ports/80x86/linux/gcc/spy/libqs.a.

2.5 Building the Examples

The standard QS distribution comes with two examples: the QHsmTst test example described in Section 2.6 of the “[QEP/C Programmer's Manual](#)”, and the QDPP example described in Section 3 of the “[QF/C Programmer's Manual](#)”. Both these applications are compiled with Turbo C++ 1.01 and run on a Windows-based PC.

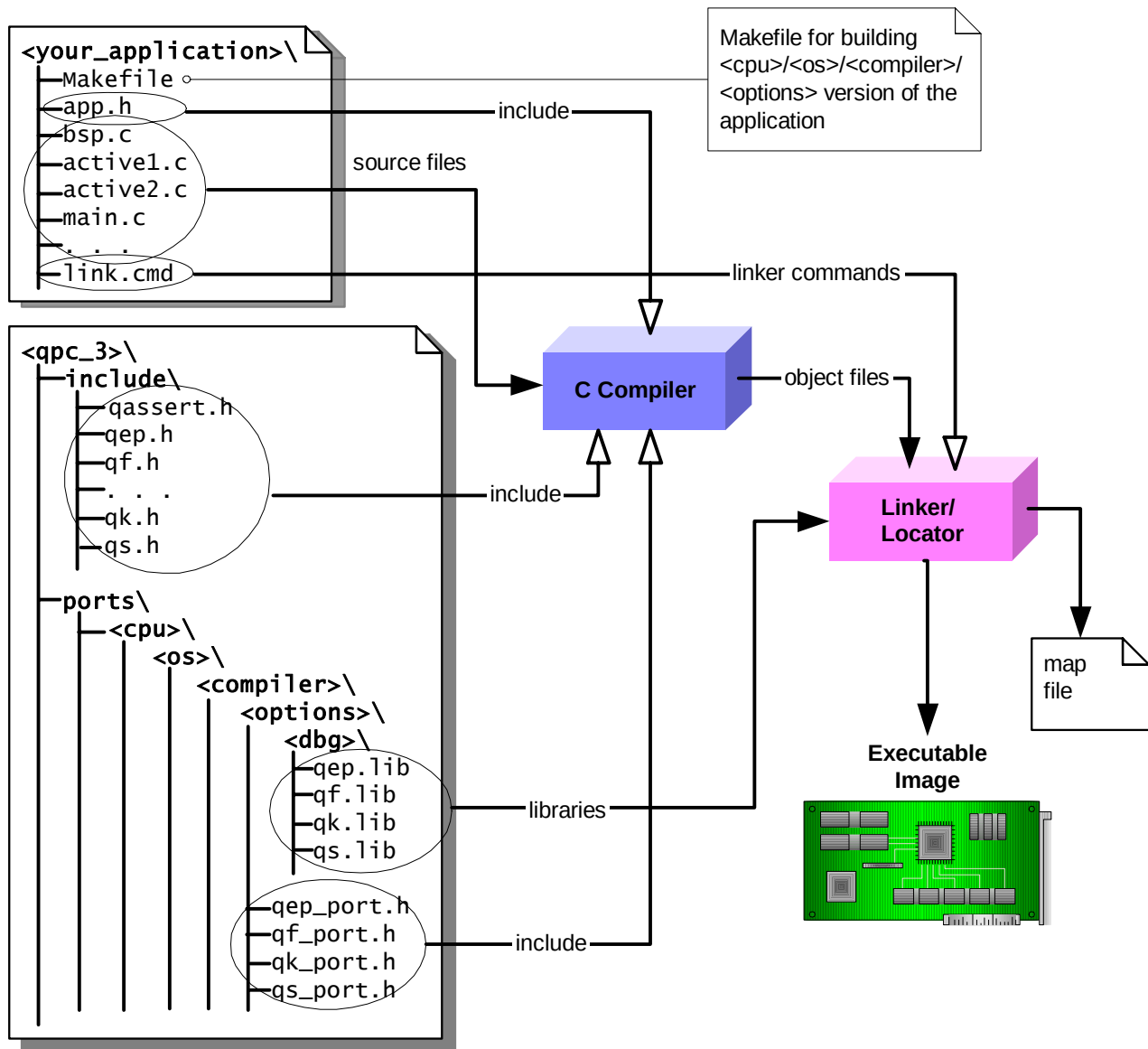


Figure 4 QS application build process

Figure 4 illustrates the general steps required to build a QS application, such as the QDPP example instrumented with QS. Please note that the C compiler includes the header files from three directories (the include directories are typically specified with the `-I` compiler option): the application directory `<your application>`, the `<qpc_3>\include\` directory and the QP port directory `<qpc_3>\ports\<cpu>\<os>\<compiler>\<options>\`. The names of subdirectories in the QP port directly indicate the choice of the CPU, operating system, compiler, and compiler options, respectively.

2.5.1 Building the QHsmTst Example with QS with Turbo C++ 1.01

The Makefile for the QHsmTst example with QS is located in: <qpc_3>\examples\80x86\dos\tcpp101\1\qhsm-spy\. To build the example for Quantum Spy, you change to this directory and invoke the make utility by typing at the command prompt the following command:

```
mingw32-make spy
```

The make utility processes the make file Makefile in the current directory to build the executable. The make file and the Turbo Linker response file <qpc_3>\examples\80x86\dos\tcpp101\1\qhsm-spy\link_spy.rsp assume that Borland Turbo C++ 1.01 has been installed in the directory C:\tools\tcpp101. You need to adjust these two elements if you've installed Turbo C++ 1.01 into a different directory.

NOTE: The legacy Turbo C++ 1.01 compiler is still subject to the DOS command-line limitation to 128 characters. The "linker response file" is a workaround. The file contains lengthy parts of commands that would not otherwise fit within the DOS limit.

2.5.2 Building the QDPP Example with QS with Turbo C++ 1.01

The Makefile for the QDPP example with QS is located in: <qpc_3>\examples\80x86\qk\tcpp101\1\qdpp-spy\. To build the example, you change to this directory and invoke the make utility by typing at the command prompt the following command:

```
mingw32-make spy
```

The make utility processes the make file Makefile in the current directory to build the executable. The make file and the Turbo Linker response file <qpc_3>\examples\80x86\qk\tcpp101\1\qdpp-spy\link_spy.rsp assume that Borland Turbo C++ 1.01 has been installed in the directory C:\tools\tcpp101. You need to adjust these two elements if you've installed Turbo C++ 1.01 into a different directory.

NOTE: The QDPP-Spy example illustrates QS instrumentation of all QP components: QEP, QF, and QK. Therefore, you need to install all these components before building the QDPP-Spy example.

2.5.3 Building The QDPP Example with QS for Linux

The make file for the Linux version of the QDPP-Spy application is located in: <qpc_3>/examples/80x86/**linux**/gcc/qdpp-spy/Makefile. To build the QS test example, you change directory to <qpc_3>/examples/80x86/**linux**/gcc/qdpp-spy and invoke the make utility by typing at the command prompt the following command:

```
make spy
```

The make process should create the application executable in the directory <qpc_3>/examples/80x86/**linux**/gcc/qdpp-spy/spy. The executable is a console application that you can execute from any Linux terminal.

2.6 The QHsmTst Example

The QHsmTst example has been described in Section 2.6 of the “[QEP/C Programmer's Manual](#)”. This example implements a hypothetical hierarchical state machine shown in Figure 5, which has been specifically designed to exhaustively test the QEP component. Here, the QEP instrumentation has been enabled by linking the application with the “spy” version of the QEP library. To avoid at first the complications of transmitting the trace data between the target and the host, this example demonstrates the simplest case, in which the trace output is sent directly to a file.

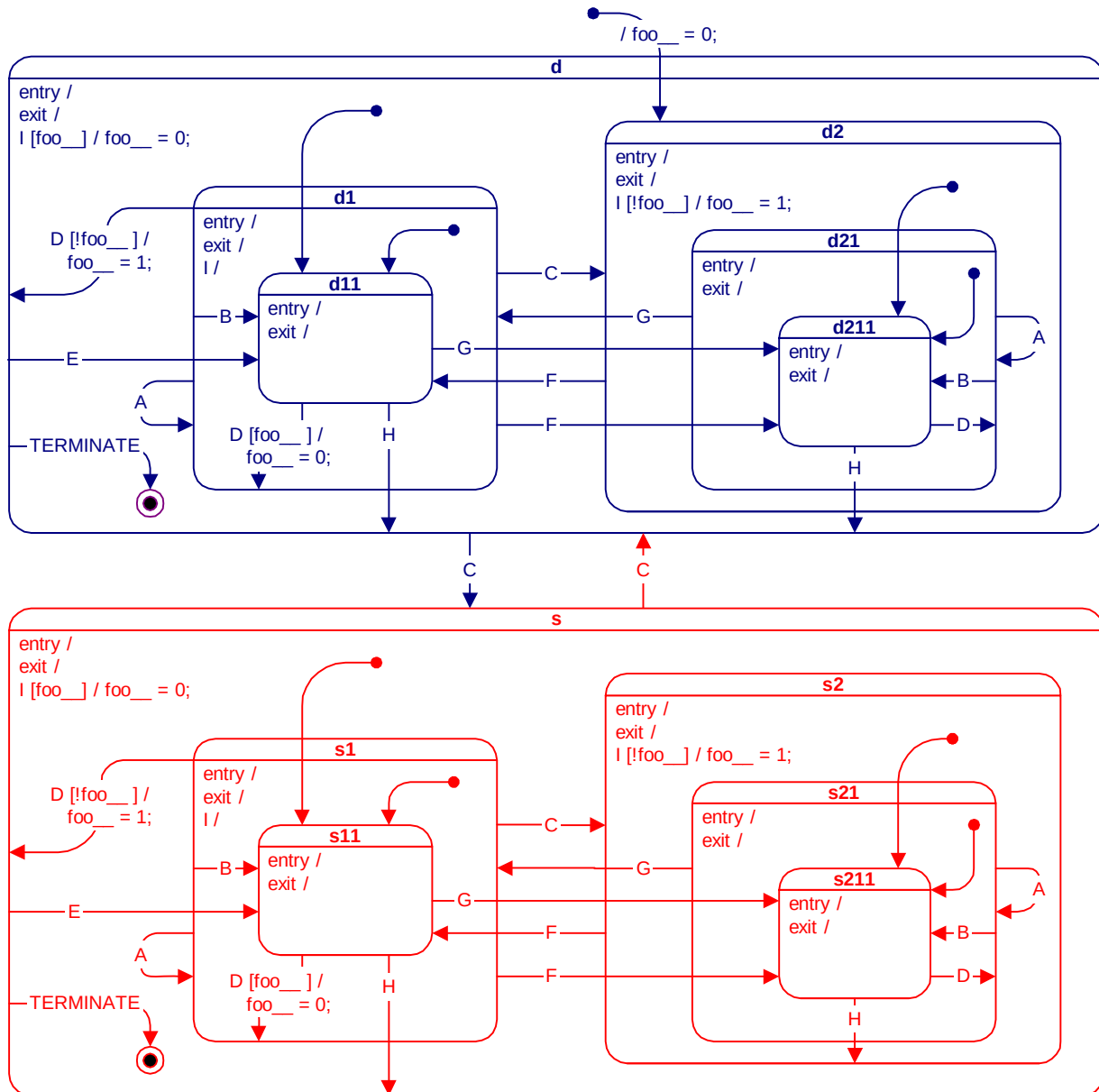


Figure 5 Hypothetical statechart used in the QHsmTst example.

2.6.1 Executing the QHsmTst Example

As described in Section 2.6 of the ["QEP/C Programmer's Manual"](#), you can run the QEP test interactively as well as in the batch mode, in which a predetermined sequence of events is dispatched to the state machine from Figure 5 and the output is collected into a file. Here, we'll use the batch mode.

You launch the QHsmTst example in the batch mode by providing the output file name as a command-line parameter. In a command prompt window, change to the directory to <qpc_3>\examples\80x86\dos\tcpp101\1\qhsm-spy\, and type:

```
spy\qhsmtst log.txt
```

The program should produce two files: (1) the textual log file log.txt described in Section 2.6.2 of the ["QEP/C Programmer's Manual"](#), and (2) the binary trace output file qs.bin.

The binary trace output qs.bin contains the QS trace data collected from the batch run of the application. You can use the QSpy host application to decompress the data and convert it to human-readable format. The Windows version of this application is located in the directory <qpc_3>\tools\qspy\win32\vc6\release\qspy.exe. The Linux version of this application is located in the directory <qpc_3>\tools\qspy\linux\g++\rel\qspy. On any host machine (Windows or Linux), you invoke the QSpy host application in the following way:

```
qspy -f qs.bin
```

This will dump the human-readable trace data to the screen. Obviously, you always can redirect this output to a file, like this:

```
qspy -f qs.bin > qs.txt
```

```
. . .
1:      Q_INIT : Obj=l_qhsmTst Source=QHsm_top Target=QHsmTst_d2
2:      Q_ENTRY: Obj=l_qhsmTst State=QHsmTst_d
3:      Q_ENTRY: Obj=l_qhsmTst State=QHsmTst_d2
4:      Q_INIT : Obj=l_qhsmTst Source=QHsmTst_d2 Target=QHsmTst_d211
5:      Q_ENTRY: Obj=l_qhsmTst State=QHsmTst_d211
6:      Q_ENTRY: Obj=l_qhsmTst State=QHsmTst_d211
7: 0000000000 ==>Init: Obj=l_qhsmTst New=QHsmTst_d211
8:      Q_EXIT : Obj=l_qhsmTst State=QHsmTst_d211
9:      Q_EXIT : Obj=l_qhsmTst State=QHsmTst_d21
10:     Q_ENTRY: Obj=l_qhsmTst State=QHsmTst_d21
11:     Q_INIT : Obj=l_qhsmTst Source=QHsmTst_d21 Target=QHsmTst_d211
12:     Q_ENTRY: Obj=l_qhsmTst State=QHsmTst_d211
13: 0000000000 ==>Tran: Obj=l_qhsmTst Sig=A_SIG Source=QHsmTst_d211 New=QHsmTst_d211
. . .
```

Listing 2 Snippet from the human-readable trace file.

Listing 2 shows a snippet from the trace data corresponding to the top-most initial transition in the QHsmTst state machine (lines 1-7) as well as handling of the first event A_SIG injected into the state machine (lines 8-13). The first column in lines 7 and 13 contains the timestamps of the initialization and transition, respectively. In the batch mode they are both zero, because the whole example completes within one time tick.

You could correlate the trace data with the corresponding output from the log.txt file, also generated in the batch mode:

```
. . .
top-INIT;d-ENTRY;d2-ENTRY;d2-INIT;d21-ENTRY;d211-ENTRY;
A:d21-A;d211-EXIT;d21-EXIT;d21-ENTRY;d21-INIT;d211-ENTRY;
```


2.6.2 Setting-up the QHsmTst Application for Tracing

Adding the QS tracing to an application requires the following three steps: (1) defining the platform-specific elements of the QS such as the trace data transfer method, (2) initializing and configuring QS, and (3) inserting the special "dictionary" records to help generate a more user-friendly trace output.

NOTE: The dictionary records provide symbolic information to the host tracing component, very much like the symbolic information included in the object files provides symbolic information to the traditional single-step debugger. The upcoming Section 0 covers the dictionary entries in detail.

```

1: #include "qep_port.h"
2: #include "qs_port.h" /* QS instrumentation interface */
3: #include "qhsmtst.h"
4:
5: /* local objects -----*/
6: static QHsmTst l_qhsmTst;
7:
8: /* Q_SPY -----*/
9: #ifdef Q_SPY
10: enum QSQHsmTstRecords {
11:     QS_QHSMTST_DISPLAY = QS_USER
12: };
13: static FILE *l_qsFile; /* file for collecting the QS trace */
14: #define QS_BUF_SIZE 256
15: /* -----*/
16: uint8_t QS_init(void const *arg) {
17:     static uint8_t qsBuf[QS_BUF_SIZE]; /* buffer for Quantum Spy */
18:     QS_initBuf(qsBuf, sizeof(qsBuf));
19:     l_qsFile = fopen((char const *)arg, "wb"); /* write binary */
20:     return (uint8_t)(l_qsFile != (FILE *)0); /* the status of the file */
21: }
22: /* -----*/
23: void QS_exit(void) {
24:     fclose(l_qsFile); /* cleanup QS */
25: }
26: /* -----*/
27: QSTimeCtr QS_getTime(void) {
28:     return (QSTimeCtr)clock();
29: }
30: /* -----*/
31: #define QS_DUMP() do { \
32:     uint16_t nBytes = QS_BUF_SIZE; \
33:     uint8_t const *block = QS_getBlock(&nBytes); \
34:     if (block != (uint8_t *)0) { \
35:         fwrite(block, 1, nBytes, l_qsFile); \
36:     } \
37: } while (0)
38: /* -----*/
39: void QS_flush() {
40:     uint16_t nBytes = QS_BUF_SIZE;
41:     uint8_t const *block;
42:     while ((block = QS_getBlock(&nBytes)) != (uint8_t *)0) {
43:         fwrite(block, 1, nBytes, l_qsFile);
44:         nBytes = QS_BUF_SIZE;
45:     }
46: }
47: #endif /* Q_SPY */
48: /* -----*/
49: int main(int argc, char *argv[]) {

```

```

50:     if (!QS_INIT("qs.bin")) {                               /* initialize the QS */
51:         printf("Unable to open QSpy file\n");
52:         return -1;
53:     }
54:     QS_FILTER_ON(QS_ALL_RECORDS);                             /* setup the QS filters... */
55:     QS_FILTER_OFF(QS_QHSMTST_DISPLAY);
56:                                     /* generate the QS dictionaries... */
57:     QS_OBJ_DICTIONARY(&l_qhsmTst);
58:     QS_SIG_DICTIONARY(A_SIG, &l_qhsmTst);
59:     QS_SIG_DICTIONARY(B_SIG, &l_qhsmTst);
60:     QS_SIG_DICTIONARY(C_SIG, &l_qhsmTst);
61:     QS_SIG_DICTIONARY(D_SIG, &l_qhsmTst);
62:     QS_SIG_DICTIONARY(E_SIG, &l_qhsmTst);
63:     QS_SIG_DICTIONARY(F_SIG, &l_qhsmTst);
64:     QS_SIG_DICTIONARY(G_SIG, &l_qhsmTst);
65:     QS_SIG_DICTIONARY(H_SIG, &l_qhsmTst);
66:     QS_SIG_DICTIONARY(I_SIG, &l_qhsmTst);
67:     QS_FUN_DICTIONARY(&QHsm_top);
68:     QS_FUN_DICTIONARY(&QHsmTst_initial);
69:     QS_FUN_DICTIONARY(&QHsmTst_d);
70:     QS_FUN_DICTIONARY(&QHsmTst_d1);
71:     QS_FUN_DICTIONARY(&QHsmTst_d11);
72:     QS_FUN_DICTIONARY(&QHsmTst_d2);
73:     QS_FUN_DICTIONARY(&QHsmTst_d21);
74:     QS_FUN_DICTIONARY(&QHsmTst_d211);
75:     QS_FUN_DICTIONARY(&QHsmTst_s);
76:     QS_FUN_DICTIONARY(&QHsmTst_s1);
77:     QS_FUN_DICTIONARY(&QHsmTst_s11);
78:     QS_FUN_DICTIONARY(&QHsmTst_s2);
79:     QS_FUN_DICTIONARY(&QHsmTst_s21);
80:     QS_FUN_DICTIONARY(&QHsmTst_s211);
81:
82:     . . .                                     /* dispatch events to the l_qhsmTst state machine */
83:
84:     QS_EXIT();                                               /* cleanup the QS */
85:     return 0;
86: }
87: /*.....*/
88: static void dispatch(QSignal sig) {
89:     QEvent e;
90:     e.sig = sig;
91:     fprintf(l_outFile, "\n%c:", (sig <= I_SIG) ? 'A' + sig - A_SIG : '?');
92:     QHsm_dispatch((QHsm *)&l_qhsmTst, &e);                /* dispatch the event */
93:     QS_DUMP();                                              /* dump the QS buffer */
94: }
95: /*.....*/
96: void QHsmTst_display(QHsmTst *me, char const *msg) {
97:     (void)me;
98:     fprintf(l_outFile, msg);
99:     QS_BEGIN(QS_QHSMTST_DISPLAY, me);                      /* application-specific QS record */
100:         QS_STR(msg);
101:     QS_END();
102: }
103: . . .

```

Listing 3 Setting up the QS in main.c (<qpc_3>\examples\80x86\dos\tcpp101\l\qhsm-spy\main.c)

Listing 3 shows the QS setup for the QHsmTst example. The QS instrumentation is added in a configurable way, so that it's only active when the macro Q_SPY is defined (see line 9). When the Q_SPY macro is not defined, the qs_port.h header file (line 2) provides dummy definitions of all QS facilities, which prevents any QS code generation when the QS instrumentation is left in the code.

Lines 9-47 of Listing 3 show the QS platform-specific initialization, cleanup, time-stamping, and trace dumping facilities. The function `QS_init()` (lines 16-21) initializes the QS. You must provide to it a contiguous memory block for the trace buffer. QS can work with a buffer of any size, but smaller buffers will lose data if the buffer "wraps around". You will always know, however, when any data loss occurs (see Section 3.8).

The macro `QS_DUMP()` (Listing 3, lines 31-37) defines how the QS data is dumped from the target to the host. In this example, the trace data is dumped to a binary file. This example illustrates the block-interface to the QS output buffer. The function `QS_getBlock()` returns the contiguous block of memory of up to `nBytes` long, if there is any QS data, or a `NULL` pointer if the QS buffer is empty. The actual number of bytes available in the data block is returned in the `nBytes` parameter.

The function `QS_flush()` (Listing 3, lines 39-46) defines how to completely flush the QS data buffer from the target to the host. Typically `QS_flush()` busy-waits until the transmission completes. However, this function is called only to output the QS "dictionary" records (see Section 0), which is typically done during the system initialization when real-time constraints don't apply yet. It can always define this function to be empty, in which case the QS buffer will be transmitted in the standard way defined by the macro `QS_DUMP()`.

The `main()` function starts off with the QS initialization, which always has to invoke the `QS_INIT()` macro (Listing 3, line 50). The `QS_INIT()` macro takes a `void*` parameter, which in this case is the string denoting the name of the trace file. The macro returns zero if the initialization fails and non-zero if it was successful.

In lines 54-55 you see a basic QS filter configuration. The default state of the QS filters after initialization is to disable all records. Therefore, the macro `QS_FILTER_ON()` in line 54 opens the filter for tracing all QS records. The next macro `QS_FILTER_OFF()` in line 55 closes the filter for the specified application-specific record `QS_QHSMST_DISPLAY`, which has been defined in line 11. You can delete line 55, if you want to trace this record defined in function `QHsmTst_display()` in lines 99-101 of Listing 3.

Lines 57-80 generate various "dictionary" trace entries to help produce a more user-friendly trace output. Such dictionary entries define a mapping between various addresses in the code and symbolic names. The dictionary entries are not absolutely required, and the QS tracing will still work without them. However, the QSpy host application will only be able to display hexadecimal addresses of state machine objects, event signals, and state-handler functions, instead using their symbolic names. The upcoming Section 0 covers the dictionary entries in detail.

In line 84 of Listing 3, you see the cleanup of QS through the macro `QS_EXIT()`. It's always a good practice to invoke this macro upon exit from the application.

Finally, in lines 99-101, you see how to generate an application-specific QS record. This record is initially filtered out (disabled) by the macro `QS_FILTER_OFF(QS_QHSMST_DISPLAY)` in line 55. If you enable this record (by deleting line 55), you will see the following additional trace records in the QS log. The upcoming Section 3.6 describes the generation of the application-specific trace records in detail.

```
. . .
0000000000 User000: top-INIT;
0000000000 User000: d-ENTRY;
0000000000 User000: d2-ENTRY;
. . .
0000000000 User000: d21-INIT;
0000000000 User000: d211-ENTRY;
. . .
```

2.7 The QDPP-SPY Example for Turbo C++ 1.01

The QDPP example has been described in Section 3 of the "QF/C Programmer's Manual". This application demonstrates the use of QS with all QP components: QEP, QF, and QK. Here, the QS instrumentation has been enabled by linking the application with the "spy" versions of the QEP, QF, and QK libraries. The QDPP-SPY example also demonstrates the QS data uplink through a serial port.

2.7.1 Executing the QDPP-SPY Example

As shown in Figure 6 to the right, you need a x86 target PC running any version of Windows or DOS, and a host machine, which can be running either Windows or Linux. You'll also need a NULL-modem serial cable to connect the target and the host.

Alternatively, if you have two serial ports in your Windows PC, you could execute both the target application, and the host QSpy application on the same machine.

First, you should launch the QSpy host application. The Windows version of this application is located in the directory <qpc_3>\tools\qspy\win32\vc6\release\qspy.exe. You invoke the QSpy host application in the following way:

```
qspy -c COM1 -b 115200
```

Please note that through the -c option you can specify a different COM port number, to which you have connected the NULL-modem cable from the target.

The Linux version of the QSpy application is located in the directory <qpc_3>/tools/qspy/linux/g++-rel/qspy. On Linux, you invoke the QSpy host application in the following way:

```
qspy -c /dev/ttyS0 -b 115200
```

Please note that you should use the serial device number /dev/ttySx to which you have connected the NULL-modem cable from the target.

The QSpy application should produce the following output to the screen:

```
QSpy 3.1.05  
Wed Jan 25 09:36:51 2006
```

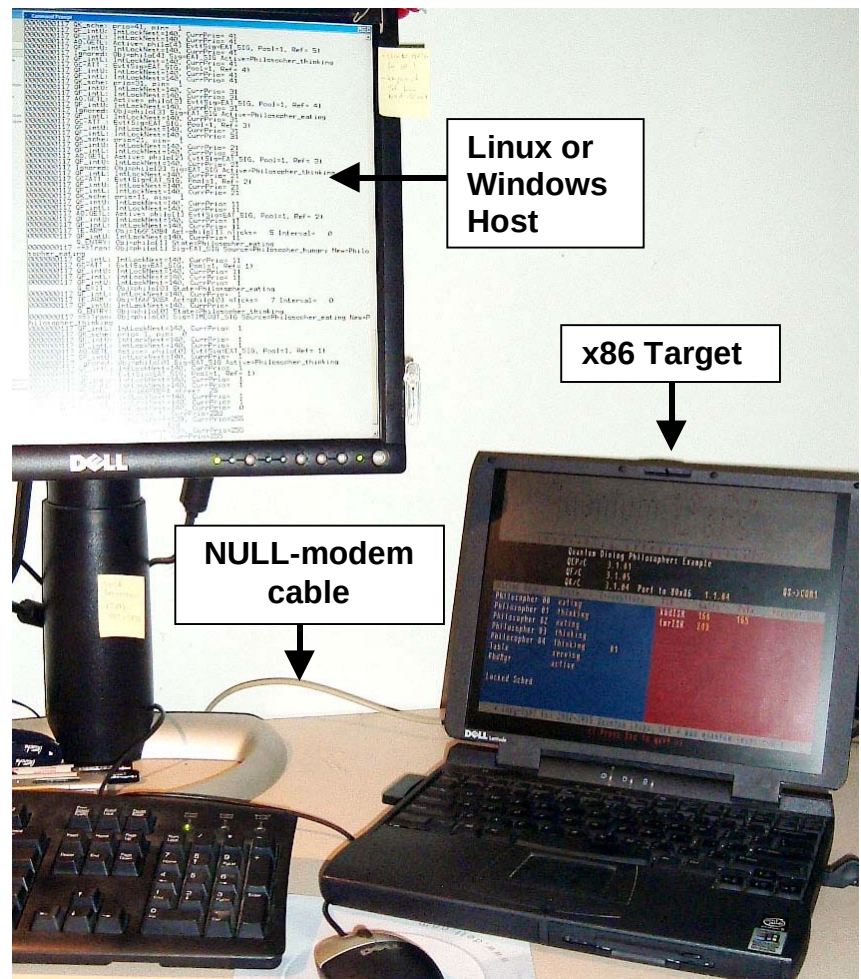


Figure 6 QDPP-SPY example running on a x86 target and sending data to a Linux Host.


```
-c /dev/ttyS0
-b 115200

Serial port COM1 opened, hit any key to stop...
```

Next, you can start the QDPP-SPY application on the target x86 machine. The executable is provided in the standard QS distribution in the directory <qpc_3>\examples\80x86\qk\tcpp101\1\qdpp-spy\spy\qdpp.exe. You can launch the executable by simply double-clicking on it, in which case the QS trace output will be sent to the COM1 serial port, which is the default. Alternatively, you can launch the example from a command prompt, in which case you can provide a COM port as a parameter.

```
<qpc_3>\examples\80x86\qk\tcpp101\1\qdpp-spy\spy\qdpp.exe COM2
```

The valid serial port selections are COM1, COM2, COM3, and COM4.

NOTE: The QDPP-SPY target application has been tested only on x86 PC's equipped with a genuine 16550-compatible UART and might not work correctly with virtual COM ports provided through a USB hub or other such solutions.

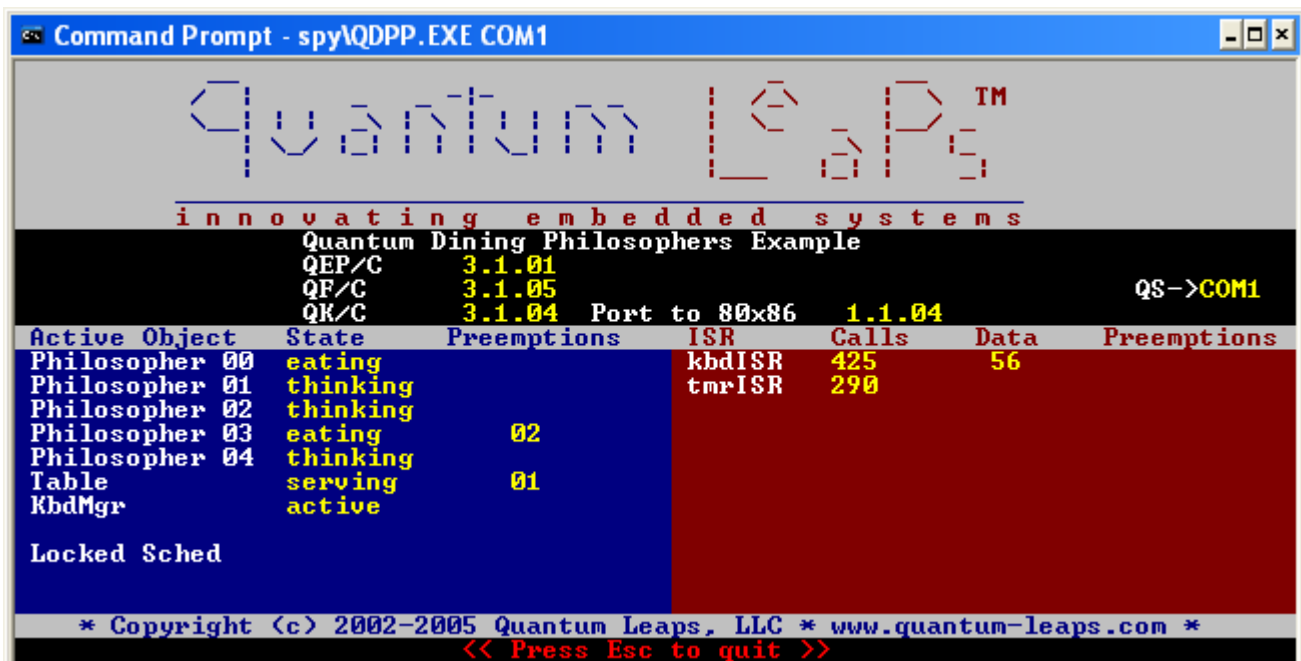


Figure 7 QDPP-SPY running in the DOS console.

After you launch the application, the QSpy host application should start produce the output to the screen. The following Listing 4 shows some fragments of the QS trace log.

```
QSpy 3.1.05
Wed Jan 25 14:35:49 2006
-c COM1
-b 115200

Serial port COM1 opened, hit any key to stop...
```

```

Obj Dic: 196B1C74->l_regPoolSto
Obj Dic: 196B1DB4->l_kbdmgrQueueSto
Obj Dic: 196B1DA0->l_tableQueueSto
Obj Dic: 196B1CD8->l_philQueueSto[0]
...
0000013397 AO.ADD : Active=phil0[0] Prio= 1
EQ.INIT: Obj=l_philQueueSto[0] Len=10
0000013425 QF_intL: IntLockNest= 1
0000013439 AO.SUB : Active=phil0[0] Sig=EAT_SIG
0000013457 QF_intU: IntLockNest= 1
Q_INIT : Obj=phil0[0] Source=QHsm_top Target=Philosopher_thinking
0000013489 QF_intL: IntLockNest= 1
0000013503 TE.ARM : Obj=phil0[0].timeEvt__ Act=phil0[0] nTicks= 17 Interval= 0
0000013523 QF_intU: IntLockNest= 1
Q_ENTRY: Obj=phil0[0] State=Philosopher_thinking
0000013545 ==>Init: Obj=phil0[0] New=Philosopher_thinking
...
Obj Dic: 196B1E90->table
Fun Dic: 15570112->Table_initial
Fun Dic: 15570194->Table_serving
0000035209 AO.ADD : Active=table Prio=51
EQ.INIT: Obj=l_tableQueueSto Len= 5
0000035233 QF_intL: IntLockNest= 1
0000035247 AO.SUB : Active=table Sig=HUNGRY_SIG
0000035263 QF_intU: IntLockNest= 1
0000035277 QF_intL: IntLockNest= 1
0000035291 AO.SUB : Active=table Sig=DONE_SIG
0000035307 QF_intU: IntLockNest= 1
0000035321 QF_intL: IntLockNest= 1
0000035335 AO.SUB : Active=table Sig=TERMINATE_SIG
0000035351 QF_intU: IntLockNest= 1
Q_INIT : Obj=table Source=QHsm_top Target=Table_serving
0000035381 ==>Init: Obj=table New=Table_serving
Obj Dic: 196B1EC0->kbdMgr
Fun Dic: 15A00106->kbdMgr_initial
Fun Dic: 15A0012A->kbdMgr_active
0000078641 AO.ADD : Active=kbdMgr Prio=61
EQ.INIT: Obj=l_kbdmgrQueueSto Len= 2
0000078667 QF_intL: IntLockNest= 1
0000078681 AO.SUB : Active=kbdMgr Sig=KBD_SIG
0000078697 QF_intU: IntLockNest= 1
Q_INIT : Obj=kbdMgr Source=QHsm_top Target=kbdMgr_active
0000078725 ==>Init: Obj=kbdMgr New=kbdMgr_active
...
0000065561 QF_isrE: IsrNest= 1, CurrPrio=255
TICK : Ctr= 1
0000065583 QF_intL: IntLockNest= 1
0000065599 QF_isrX: IsrNest= 1, CurrPrio=255
0000065613 QF_intU: IntLockNest= 1
0000131097 QF_isrE: IsrNest= 1, CurrPrio=255
TICK : Ctr= 2
0000131117 QF_intL: IntLockNest= 1
0000131133 QF_isrX: IsrNest= 1, CurrPrio=255
0000131149 QF_intU: IntLockNest= 1
...
0001114131 QF_isrE: IsrNest= 1, CurrPrio=255
TICK : Ctr= 17
TE.ADRM: Obj=phil0[4].timeEvt__ Act=phil0[4]
0001114157 TE.POST: Obj=phil0[4].timeEvt__ Sig=TIMEOUT_SIG Act=phil0[4]
0001114179 QF_intL: IntLockNest= 1
0001114193 AO.FIFO: Obj=phil0[4] Evt(Sig=TIMEOUT_SIG, Pool=0, Ref= 0) Queue(nUsed= 0, nMax= 0)
0001114213 QF_intU: IntLockNest= 1
TE.ADRM: Obj=phil0[3].timeEvt__ Act=phil0[3]

```

```

0001114235 TE.POST: Obj=philo[3].timeEvt__ Sig=TIMEOUT_SIG Act=philo[3]
0001114255 QF_intL: IntLockNest= 1
0001114269 AO.FIFO: Obj=philo[3] Evt(Sig=TIMEOUT_SIG, Pool=0, Ref= 0) Queue(nUsed= 0, nMax=
0)
0001114289 QF_intU: IntLockNest= 1
...
0001114523 QF_intL: IntLockNest= 1
0001114539 QF_isrX: IsrNest= 1, CurrPrio=255
0001114557 QK_sche: prio=41, pin= 0
0001114573 QF_intU: IntLockNest= 1
0001114585 QF_intL: IntLockNest= 1
0001114603 AO.GETL: Active= philo[4] Evt(Sig=TIMEOUT_SIG, Pool=0, Ref= 0)
0001114621 QF_intU: IntLockNest= 1
0001114659 NEW : Evt(Sig=HUNGRY_SIG, size= 3)
0001114677 QF_intL: IntLockNest= 1
0001114691 MP.GET : Obj=l_regPoolSto nFree= 24 nMin= 24
0001114709 QF_intU: IntLockNest= 1
0001114723 QF_intL: IntLockNest= 1
0001114737 QF_intU: IntLockNest= 1
0001114753 QF_intL: IntLockNest= 1
0001114765 AO.FIFO: Obj=table Evt(Sig=HUNGRY_SIG, Pool=1, Ref= 1) Queue(nUsed= 0, nMax= 0)
0001114787 QK_sche: prio=51, pin= 41
0001114799 QF_intU: IntLockNest= 1
0001114813 QF_intL: IntLockNest= 1
0001114829 AO.GETL: Active= table Evt(Sig=HUNGRY_SIG, Pool=1, Ref= 1)
0001114845 QF_intU: IntLockNest= 1
0001114905 NEW : Evt(Sig=EAT_SIG, size= 3)
0001114919 QF_intL: IntLockNest= 1
0001114933 MP.GET : Obj=l_regPoolSto nFree= 23 nMin= 23
0001114949 QF_intU: IntLockNest= 1
0001114967 QF_intL: IntLockNest= 1
0001114981 QK_schL: SchedLockNest= 1, CurrPrio= 64
0001114995 QF_intU: IntLockNest= 1
0001115011 QF_intL: IntLockNest= 1
0001115025 AO.FIFO: Obj=philo[4] Evt(Sig=EAT_SIG, Pool=1, Ref= 0) Queue(nUsed= 0, nMax= 0)
0001115045 QF_intU: IntLockNest= 1
0001115061 QF_intL: IntLockNest= 1
0001115075 AO.FIFO: Obj=philo[3] Evt(Sig=EAT_SIG, Pool=1, Ref= 0) Queue(nUsed= 0, nMax= 0)
0001115097 QF_intU: IntLockNest= 1
0001115113 QF_intL: IntLockNest= 1
0001115127 AO.FIFO: Obj=philo[2] Evt(Sig=EAT_SIG, Pool=1, Ref= 0) Queue(nUsed= 0, nMax= 0)
0001115147 QF_intU: IntLockNest= 1
0001115161 QF_intL: IntLockNest= 1
0001115175 AO.FIFO: Obj=philo[1] Evt(Sig=EAT_SIG, Pool=1, Ref= 0) Queue(nUsed= 0, nMax= 0)
0001115195 QF_intU: IntLockNest= 1
0001115211 QF_intL: IntLockNest= 1
0001115223 AO.FIFO: Obj=philo[0] Evt(Sig=EAT_SIG, Pool=1, Ref= 0) Queue(nUsed= 0, nMax= 0)
0001115243 QF_intU: IntLockNest= 1
0001115257 PUBLISH: Evt(Sig=EAT_SIG, Pool=1, Ref= 5) nSubsr= 5
0001115273 QF_intL: IntLockNest= 1
0001115289 QK_schu: SchedLockNest= 1, CurrPrio= 64
0001115303 QF_intU: IntLockNest= 1
0001115359 Intern : Obj=table Sig=HUNGRY_SIG Source=Table_serving
0001115381 QF_intL: IntLockNest= 1
0001115395 GC : Evt(Sig=HUNGRY_SIG, Pool=1, Ref= 1)
0001115409 QF_intU: IntLockNest= 1
0001115423 QF_intL: IntLockNest= 1
0001115439 MP.PUT : Obj=l_regPoolSto nFree= 24
0001115455 QF_intU: IntLockNest= 1
...
0001119629 QF_intL: IntLockNest= 1
0001119643 GC : Evt(Sig=EAT_SIG, Pool=1, Ref= 1)
0001119657 QF_intU: IntLockNest= 1
0001119671 QF_intL: IntLockNest= 1

```



```
0001119685 MP.PUT : Obj=l_regPoolSto nFree= 25
0001119701 QF_intU: IntLockNest= 1
0001119715 QF_intL: IntLockNest= 1
0001119729 QF_intU: IntLockNest= 1
0001179671 QF_isrE: IsrNest= 1, CurrPrio=255
          TICK   : Ctr=      18
```

Listing 4 Sample QS trace output from QDPP-SPY with all trace records enabled.

The fragments of the QS trace log shown in Listing 4 contain very detailed information, because all QS records are enabled. The following bullet items highlight the most interesting parts of the trace and illustrate how you can interpret the trace data:

- The QS log always contains the QSpy application version number, the date and time of the run, and all the options explicitly provided to the QSpy application.
- A log typically starts with the dictionary records that provide a mapping between addresses of various objects and state-handler functions and their symbolic names. The dictionary entries don't have timestamps.
- After the dictionaries, you see the active object initialization, for example at time stamp 0000013397, you see adding the philo[0] object with priority 1 and initializing the event queue of length 10. This is followed by the initial transition that subscribes to the EAT_SIG. and arming the philo[0].timeEvt__ time event in 17 ticks for one shot. The initial transition ends in the Philosopher_thinking state at the timestamp 0000013545.
- Active object initialization can contain also contain dictionary entries for items that are encapsulated within the active object. For example initialization of the Table active object insets two function dictionary entries for state handlers Table_initial and Table_serving. Please also note that typically the dictionary data is flushed right away to the host, but this takes time (compare timestamps before and after the dictionary entries.)
- After the active object initialization, interrupts are enabled, and the first Tick interrupt arrives at the timestamp 0000065561. You can determine the type of the interrupt by the unique priority number. For example, the priority of the Tick interrupt is 0xFF == 255.
- The Tick interrupt occurs for 17 times. You can determine the ticking rate by comparing the time stamps between Tick 1 and 2, which is (0000131097 - 0000065561 == 65536). In the case of the QDPP application, the time stamp is provided from counter-0 of the 8254 timer/counter, which is driven from an oscillator running at 1.193182MHz. The same counter-0 of the 8254 also generates time tick interrupts every 0x10000 number of counts (the famous 18.2Hz DOS tick).
- In the Tick 17 at timestamp 0001114131, you see that time events post TIMEOUT_SIG events to the Philosopher active objects. This triggers a lot of activity in the application. In fact, over 340 trace records occur before the next Tick 18.

NOTE: The QSpyTM component is implemented as a portable console application. The application does not have any fancy GUI, because its function is to provide only the QS data parsing, storing, and exporting to such powerful tools as MATLAB®.

2.7.2 Initializing QDPP Application for Tracing

The QDPP-SPY example demonstrates all aspects of QS setup. In particular you'll see how to send the QS trace data over a serial port (UART) and how to time-stamp QS records with sub-microsecond precision using the 8254 timer/counter found in every x86-based PC.

```

1: #include "qf_port.h"
2: #include "qdpp.h"
3: #include "qs_port.h"
4: . . .
5: /* local objects -----*/
6: static QEvent const *_l_kbdmgrQueueSto[2];
7: static QEvent const *_l_tableQueueSto[N];
8: static QEvent const *_l_philQueueSto[N][10];
9: static void *_l_regPoolSto[100/sizeof(void*)]; /*100 bytes for the event pool */
10: static QSubscrList l_subscrSto[MAX_PUB_SIG];
11:
12: /* . . . */
13: int main(int argc, char *argv[]) {
14:     uint8_t n;
15:     char const *comName = "COM1";
16:
17:     if (argc > 1) { /* COM name specified? */
18:         comName = argv[1];
19:     }
20:
21:     if (!QS_INIT(comName)) { /* initialize QSpy */
22:         printf("Unable to open QSpy output\n");
23:         return -1;
24:     }
25:
26:     /* filters... */
27:     QS_FILTER_ON(QS_ALL_RECORDS);
28:     /* object dictionaries... */
29:     QS_OBJ_DICTIONARY(l_regPoolSto);
30:     QS_OBJ_DICTIONARY(l_kbdmgrQueueSto);
31:     QS_OBJ_DICTIONARY(l_tableQueueSto);
32:     QS_OBJ_DICTIONARY(l_philQueueSto[0]);
33:     QS_OBJ_DICTIONARY(l_philQueueSto[1]);
34:     QS_OBJ_DICTIONARY(l_philQueueSto[2]);
35:     QS_OBJ_DICTIONARY(l_philQueueSto[3]);
36:     QS_OBJ_DICTIONARY(l_philQueueSto[4]);
37:     /* function dictionaries... */
38:     QS_FUN_DICTIONARY(&QHsm_top);
39:     /* globally published signal dictionaries... */
40:     QS_SIG_DICTIONARY(HUNGRY_SIG, 0);
41:     QS_SIG_DICTIONARY(DONE_SIG, 0);
42:     QS_SIG_DICTIONARY(EAT_SIG, 0);
43:     QS_SIG_DICTIONARY(TERMINATE_SIG, 0);
44:     QS_SIG_DICTIONARY(KBD_SIG, 0);
45:     . . .
46: }
```

Listing 5 Initialization of QS, setting-up the filters, and generating dictionary entries.

Listing 5 shows the main.c module of the QDPP application. The QS initialization (line 21) and filter setup (line 26) are very straightforward. In lines 28-43, you see generation of various QS dictionary entries, which the upcoming Section 0 covers in detail.

2.7.3 Defining Platform-Specific QS Callbacks

```

1: #include "qf_port.h"
2: #include "qs_port.h"
3: #include "qdpp.h"
4: . . .
5:
6: /*-----*/
7: #ifdef Q_SPY
8: static uint16_t l_base;          /* QS data uplink UART base address */
9: /*-----*/
10: static uint8_t UART_config(char const *comName, uint32_t baud) {
11:     . . .
12: }
13: /*-----*/
14: uint8_t QS_init(void const *arg) {
15:     static uint8_t qsBuf[4*1024];          /* buffer for Quantum Spy */
16:     QS_initBuf(qsBuf, sizeof(qsBuf));
17:     return UART_config((char const *)arg, 115200UL);
18: }
19: /*-----*/
20: void QS_exit(void) {
21: }
22: /*-----*/
23: QSTimeCtr QS_getTime(void) {          /* invoked with interrupts locked */
24:     static uint32_t l_lastTime;
25:     uint32_t now;
26:     uint16_t count16;                /* 16-bit count from the 8254 */
27:     outportb(0x43, 0);              /* latch the 8254's counter-0 count */
28:     count16 = (uint16_t)inportb(0x40); /* read the low byte of counter-0 */
29:     count16 += ((uint16_t)inportb(0x40) << 8); /* add on the hi byte */
30:     now = ((uint32_t)QF_tickCtr_ << 16) + (0xFFFF - count16);
31:     if (l_lastTime > now) {          /* are we going "back" in time? */
32:         now += 0x10000;              /* assume that there was one rollover */
33:     }
34:     l_lastTime = now;
35:     return (QSTimeCtr)now;
36: }
37: /*-----*/
38: #define QS_DUMP() \
39:     if ((inportb(l_base + 5) & (1 << 5)) != 0) { \
40:         uint8_t fifo = 16; \
41:         uint16_t b; \
42:         QS_INT_LOCK(igonre); \
43:         while ((fifo != 0) && ((b = QS_getByte()) != QS_EOD)) { \
44:             QS_INT_UNLOCK(igonre); \
45:             outportb(l_base + 0, (uint8_t)b); \
46:             --fifo; \
47:             QS_INT_LOCK(igonre); \
48:         } \
49:         QS_INT_UNLOCK(igonre); \
50:     } else ((void)0)
51: /*-----*/
52: void QS_flush(void) {
53:     uint16_t b;
54:     while ((b = QS_getByte()) != QS_EOD) { /* next QS trace byte available? */
55:         while ((inportb(l_base + 5) & (1 << 5)) == 0) { /* TX FIFO not empty? */
56:             ;
57:         }
58:         outportb(l_base + 0, (uint8_t)b); /* stick the byte to the TX FIFO */
59:     }
60: }
61: #endif          /* Q_SPY */
62: . . .
63: void QK_onIdle(void) {

```

```
64:     QS_DUMP();
65: }
66: /* ..... */
67: void QF_exit(void) {
68:     QS_EXIT();                               /* cleanup the QS */
69: }
```

Listing 6 QS callbacks in the QDPP BSP

Listing 6 shows the platform-specific QS callback functions, which make up a QS port. The `qs_init()` function allocates the QS trace buffer, calls `qs_initBuf()`, and configures the selected UART for 115200 baud rate (line 18).

The callback function `qs_getTime()` uses the counter-0 of the 8254 timer/counter chip found in every IBM-compatible PC for the sub-microsecond time stamping of the QS records. The counter-0 of the 8254 chip is a 16-bit down-counter, which is typically setup to generate the standard 18.2Hz interrupt when it underflows from 0 to 0xFFFF. The down-counting rate is 1.193182MHz, which works out to approximately 0.838 microseconds/count.

The basic idea of using this 16-bit down-counter for 32-bit time-stamping of the QS records is simple. The `qf_tick()` function already counts the number of clock ticks (i.e., the number of roll-overs from 0 to 0xFFFF), and the 8254 has the remainder on chip. Hence, to construct the complete 32-bit tick count, multiply the `qf_tickCtr_` by 0x10000 and add the on-chip count, or actually 0xFFFF – the on-chip count, because it's a downcounter.

NOTE: The procedure described above is commonly used to measure sub-microsecond time intervals with a PC. However, you would need more than the once-per-period “sampling” rate to completely resolve the timer-cycle ambiguity. For the high values of the counter-0 (values close to 0xFFFF), you simply cannot tell if this is just before or just after the tick ISR has been serviced, so the tick counter `qf_tickCtr_` might or might not have been updated. This means that you can be sometimes short by one whole clock tick of 54.9 ms. As a remedy, lines 32-34 of Listing 6 contain a protection against the time going “backwards”. This is a kludge and is not completely bullet-proof, if the function `qs_getTime()` is not called for more than 1 time tick.

The macro `QS_DUMP()` in lines 39-51 of Listing 6 defines how the QS data is dumped from the target to the host. In this example, the trace data is transmitted through a 16550-compatible UART. The UART is configured to use the 16-byte deep transmit FIFO in the function `UART_config()` (line 11). Having a FIFO is advantageous because the FIFO can be loaded quickly, while the actual transmission happens automatically without CPU's attention. In this example, the `QS_DUMP()` macro demonstrates the use of the byte-oriented interface to the QS transmit buffer `qs_getByte()` in line 44 of Listing 6. The `qs_getByte()` function obtains the next trace byte from the QS trace buffer and returns it as a value of type `uint16_t`. Successive calls to `qs_getByte()` will return successive bytes from the trace buffer. If the trace buffer has no more data, the function `qs_getByte()` returns `QS_EOD` (End-Of-Data), which is defined in `qs.h` to `((uint16_t)0xFFFF)`.

NOTE: The function `qs_getByte()` does not internally use a critical section because it would be not desirable in some cases; for example inside ISRs. However, if `qs_getByte()` is used at a task level, you need to provide a QS critical section around it, as demonstrated in Listing 6.

The macro `QS_DUMP()` is called in line 64 from `qf_idle()`, which shows how to utilize the idle CPU cycles that would not be used by the application anyway. Please note that `qf_idle()` can be interrupted at any point, but typically the data transmission continues from the FIFO without the CPU's attention.

Finally, the `QS_flush()` function implements the transmission by busy-waiting until all bytes are flushed from the QS transmit buffer. As mentioned in Section 2.6.2, this function is only called after the dictionary entries, which typically are generated during the initialization sequence, even before interrupts are enabled. Therefore, this function does not even bother using the critical section around the function `QS_getByte()`.

2.7.4 Generating Dictionary Records from Active Objects

The few dictionary records generated from `main.c` (Listing 5) provide only the symbolic information available at the global level. However, the encapsulated application components, such as the `Philosopher`, the `Table`, and the `KbdMgr` active objects, also need to provide the symbolic information in the trace to the QSpy host application.

This section shows how to generate dictionary trace records from individual components without compromising their encapsulation.

```
1: #include "qf_port.h"
2: #include "qs_port.h"
3: #include "qassert.h"
4: #include "qdpp.h"
5: ...
6: /* local objects -----*/
7: static Table l_table;
8: ...
9: /* .....*/
10: void Table_initial(Table *me, QEvent const *e) {
11:     uint8_t n;
12:     (void)e;                /* suppress the compiler warning */
13:
14:     QS_OBJ_DICTIONARY(&l_table);
15:     QS_FUN_DICTIONARY(&Table_initial);
16:     QS_FUN_DICTIONARY(&Table_serving);
17:
18:     QActive_subscribe_((QActive *)me, HUNGRY_SIG);
19:     QActive_subscribe_((QActive *)me, DONE_SIG);
20:     QActive_subscribe_((QActive *)me, TERMINATE_SIG);
21:     for (n = 0; n < N; ++n) {
22:         me->fork__[n] = FREE;
23:         me->isHungry__[n] = 0;
24:     }
25:     Q_INIT(&Table_serving);
26: }
```

Listing 7 Generating dictionary records from the `Table` active object (`table.c`)

Listing 7 shows the dictionary records instrumented into the `Table` active object. Generally, initial pseudostate (see Section 3.5 in the "[QEP/C Programmer's Manual](#)") is the best place for placing the dictionary records. The QF executes the top-most initial transition directly from `QActive_start` (see Section 5.4.4 in the "[QF/C Programmer's Manual](#)"), therefore the dictionary records are generated during the initial transient, before the real-time constraints start to apply.

Please note that the object dictionary `QS_OBJ_DICTIONARY()` is generated using the address of the static variable `l_table`, rather than the generic pointer `"me"`. Obviously, the name `"l_table"` is more descriptive than `"me"`.

```
1: #include "qf_port.h"
2: #include "qs_port.h"
3: #include "qassert.h"
4: #include "qdpp.h"
```

```

5:  . . .
6:  /* local objects ----- */
7:  static Philosopher l_philo[N];
8:  . . .
9:  /* ----- */
10: void Philosopher_initial(Philosopher *me, QEvent const *e) {
11:     static uint8_t registered;      /* starts off with 0, per C-standard */
12:     if (!registered) {
13:         QS_OBJ_DICTIONARY(&l_philo[0]);
14:         QS_OBJ_DICTIONARY(&l_philo[0].timeEvt__);
15:         QS_OBJ_DICTIONARY(&l_philo[1]);
16:         QS_OBJ_DICTIONARY(&l_philo[1].timeEvt__);
17:         QS_OBJ_DICTIONARY(&l_philo[2]);
18:         QS_OBJ_DICTIONARY(&l_philo[2].timeEvt__);
19:         QS_OBJ_DICTIONARY(&l_philo[3]);
20:         QS_OBJ_DICTIONARY(&l_philo[3].timeEvt__);
21:         QS_OBJ_DICTIONARY(&l_philo[4]);
22:         QS_OBJ_DICTIONARY(&l_philo[4].timeEvt__);
23:
24:         QS_FUN_DICTIONARY(&Philosopher_initial);
25:         QS_FUN_DICTIONARY(&Philosopher_thinking);
26:         QS_FUN_DICTIONARY(&Philosopher_hungry);
27:         QS_FUN_DICTIONARY(&Philosopher_eating);
28:
29:         registered = (uint8_t)1;
30:     }
31:     QS_SIG_DICTIONARY(TIMEOUT_SIG, me);    /* state machine-specific signal */
32:
33:     me->num__ = ((TableEvt const *)e)->philNum;
34:     QActive_subscribe_((QActive *)me, EAT_SIG);
35:     Q_INIT(&Philosopher_thinking);
36: }

```

Listing 8 Generating dictionary records from the Philosopher active object (phil.c)

Listing 8 shows the dictionary entries instrumented into the Philosopher active objects. As before, the dictionary records are placed in the initial pseudostate of the Philosopher state machine. In this case, however, more than one Philosopher active objects are instantiated, so the top-most initial transition is executed multiple times for each instance of the Philosopher active object. In order to avoid repeating the dictionary entries, a static flag is used to generate the dictionaries only once.

The signal dictionary entry in line 31 of Listing 8, on the other hand, is generated for each state machine individually. This is because the TIMEOUT_SIG signal is a state machine-specific and the same numerical signal value can potentially represent different signals for each state machine. Section 0 discusses dictionary entries in detail.

2.7.5 Adding Application-Specific Trace Records

Although QS generates a very detailed trace even without adding any instrumentation to the application, it also allows inserting application-specific records into the trace. Section 3.6 explains application-specific trace records in more detail.

2.8 The QDPP-SPY Example for Linux

The QDPP-SPY example for Linux demonstrates the use of TCP/IP for sending QS trace data from the target to the host (see Figure 8). The target in this case is a Linux machine. The host could be the same Linux machine, or a separate Windows or Linux machine connected through a network to the target.

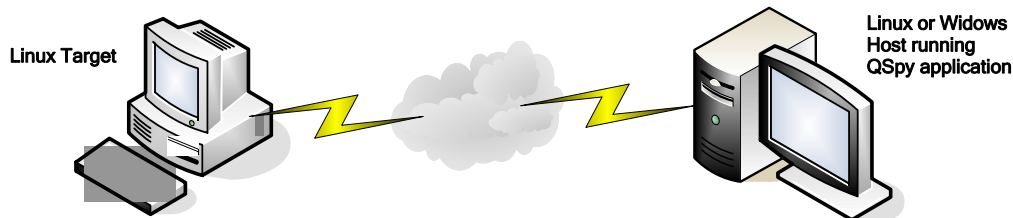


Figure 8 Software tracing with QS over the network

2.8.1 Executing the QDPP-SPY Example

First, you should launch the QSpy host application, which is the TCP/IP server for targets to connect to. The Linux version of this application is located in the directory `<qpc_3>/tools/qspy/linux/g++/dbg/qspy`. (First, you probably need to build the QSpy application for your Linux distribution. The Makefile is provided.) You invoke the QSpy host application in the following way:

```
qspy -t -S2 -E4 -Q4 -P4 -C4 -T4
```

Please note that through the `-t` option you specify the TCP/IP connection type. By default, the QSpy application opens a server socket at the port number 6601. You can override this default by the `-p` option, for example `-p 6602`. The other options have to do with the sizes of various objects in the Linux QP port, which are bigger than the defaults assumed by the QSpy application.

The QSpy application should produce the following output to the screen:

```
QSpy 3.1.05
Tue Feb 07 11:03:31 2006
-t
-S 2
-E 4
-Q 4
-P 4
-C 4
-T 4

TCP/IP port 6601 opened, hit any key to quit...
(the target must be stopped)
```

Next, you can start the QDPP-SPY application on the target Linux machine. The Makefile is provided in the standard QS distribution in the directory `<qpc_3>/examples/80x86/linux/gcc/qdpp-spy/`. You should build the SPY version of the executable by providing the "spy" target to the make:

```
make spy
```

You can now launch the qdpp executable located in the spy subdirectory. If you start it without any parameters, the default location of the QSpy server is assumed to be localhost, port 6601. You can override this default by providing one command-line argument to the qdpp application. The argument is of the form `<host>:<port>`, for example:

```
spy/qdpp myQspyServer:6601
```

At this point the QSpy host application should start showing the trace coming from the target. To stop tracing, you should quit the target application first by pressing ESC key. When the target hangs up, the QSpy application becomes ready to accept next client. Please note that QSpy can be stopped by pressing any key, but it actually exits only when not servicing a client. You should not kill the QSpy with Ctr-C in the middle of the session.

2.8.2 Defining Platform-Specific QS Callbacks

The TCP/IP example for Linux uses the same startup code as the Turbo C++ example for DOS. The only difference is in defining the QS callbacks to transfer the data over a TCP/IP socket rather than serial port. The code is located in <qpc_3>/examples/80x86/linux/gcc/qdpp-spy/bsp.c and contains very standard socket programming. The TCP/IP code uses the block-oriented QS interface, as shown in Listing 9.

```

1:
2: int l_sock = -1;
3: /* ..... */
4: uint8_t QS_init(void const *arg) {
5:     static uint8_t qsBuf[1024];          /* 1K buffer for Quantum Spy */
6:
7:     l_sock = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);    /* TCP socket */
8:
9:     server = gethostbyname(host);
10:
11:     if (connect(l_sock, (struct sockaddr *)&servAddr, sizeof(servAddr)) == -1) {
12:         QS_exit();
13:         return (uint8_t)0;
14:     }
15:     return (uint8_t)1;                    /* success */
16: }
17: /* ..... */
18: void QS_exit(void) {
19:     if (l_sock != -1) {
20:         close(l_sock);
21:     }
22: }
23: #define QS_DUMP() do { \
24:     uint16_t nBytes = 1000; \
25:     uint8_t const *block; \
26:     QS_INT_LOCK(ignore); \
27:     block = QS_getBlock(&nBytes); \
28:     QS_INT_UNLOCK(ignore); \
29:     if (block != (uint8_t *)0) { \
30:         send(l_sock, (char const *)block, nBytes, 0); \
31:     } \
32: } while (0)
33: /* ..... */
34: void QS_flush(void) {
35:     uint16_t nBytes = 1000;
36:     uint8_t const *block;
37:     while ((block = QS_getBlock(&nBytes)) != (uint8_t *)0) {
38:         send(l_sock, (char const *)block, nBytes, 0);
39:         nBytes = 1000;
40:     }
41: }

```

Listing 9 TCP/IP socket interface used for QS on Linux

3 QS Target Component

As shown in Figure 9, the QS target component consists of the ring buffer, the global and local filters, as well as the instrumentation added to QEP, QF, QK, and the Application.

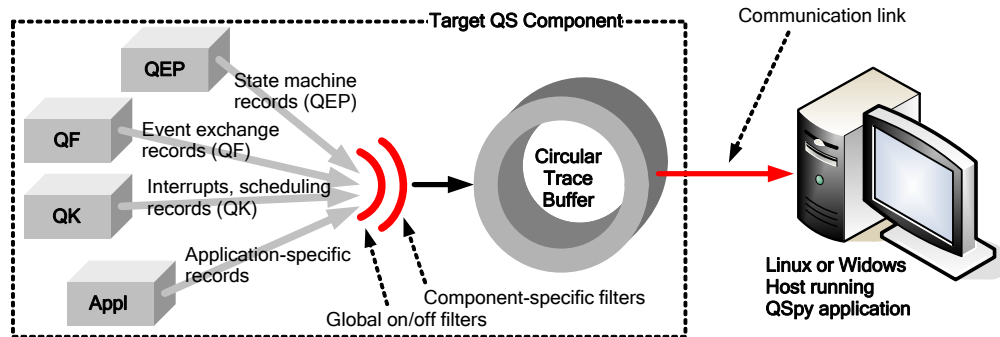


Figure 9 QS structure

3.1 QS Source Code Structure

The QS source code consists of the following files:

<code><qpc_3>/</code>	- QP-root directory for Quantum Platform (QP) v3.x.yy
<code>+--include/</code>	
<code>+ --qs.h</code>	- QS platform-independent interface
<code>+--ports/</code>	
<code>+--80x86/</code>	- QP ports to generic 80x86, both real and protected mode
<code>+--dos/</code>	- Ports for DOS
<code>+--tcpp101/</code>	- Turbo C++ 1.01 compiler
<code>+--l/</code>	- Large memory model
<code>+--qs_port.h</code>	- QS port to 80x86, DOS, Turbo C++ 1.01, Large memory model
<code>+--qk/</code>	- Ports for the QK
<code>+--tcpp101/</code>	- Turbo C++ 1.01 compiler
<code>+--l/</code>	- Large memory model
<code>+--qs_port.h</code>	- QS port to 80x86, QK, Turbo C++ 1.01, Large memory model
<code>..</code>	- other QP ports ...
<code>+--qs/</code>	- QS component (source files and ports)
<code>+--80x86/</code>	- QS ports to generic 80x86, both real and protected mode
<code>...</code>	- other QS ports ...
<code>+--source/</code>	- QS platform-independent source code (*.c files)
<code>+--qs_pkg.h</code>	- internal, packet-scope interface for the QS implementation
<code>+--qs.c</code>	- contains internal ring buffer and formatted output functions
<code>+--qs_.c</code>	- contains basic unformatted output functions
<code>+--qs_blk.c</code>	- contains block-oriented interface <code>QS_getBlock()</code>
<code>+--qs_byte.c</code>	- contains byte-oriented interface <code>QS_getByte()</code>
<code>+--qs_f32.c</code>	- contains 32-bit IEEE floating point output <code>QS_f32()</code>
<code>+--qs_f64.c</code>	- contains 64-bit IEEE floating point output <code>QS_f64()</code>
<code>+--qs_mem.c</code>	- contains memory-block output
<code>+--qs_str.c</code>	- contains zero-terminated string output

3.2 The QS Platform-Independent Header File qs.h

The platform-independent header file <qpc_3>\include\qs.h specifies the interface to all QS facilities. This header file must be included in every module that uses QS, but the inclusion is typically indirect through the platform-specific header file qs_port.h (see Section 0). As most software tracing systems for C or C++, QS instrumentation code relies heavily on the C preprocessor for the tracing to be enabled or disabled at compile time without changing the instrumented source code.

```

1: #ifndef Q_SPY                                /* Q_SPY defined--QS tracing ENABLED */
2:   #include <stdint.h>                          /* C99 exact-width integer types */
3:
4:   enum QSPyRecords {                          /* the pre-defined internal QS records */
5:       QS_QEP_STATE_EMPTY,                      /* see NOTE01 */
6:       QS_QEP_STATE_ENTRY,                      /* a state was entered */
7:       . . .
8:       QS_USER                                  /* the first record available for user QS records */
9:   };
10:
11:   extern uint8_t QS_glbFilter[32];             /* global QS filter */
12:   extern void const *QS_smObj_;                /* state machine for QEP record filter */
13:   extern void const *QS_aoObj_;                /* active object for QF record filter */
14:   extern void const *QS_mpObj_;                /* event pool for QF record filter */
15:   extern void const *QS_eqObj_;                /* raw queue for QF record filter */
16:   extern void const *QS_teObj_;                /* time event for QF record filter */
17:   extern void const *QS_apObj_;                /* generic object Application local filter */
18:
19:   uint16_t QS_getByte(void);
20:   uint8_t const *QS_getBlock(uint16_t *pNum);
21:
22:   uint8_t QS_init(void const *arg);
23:   void QS_flush(void);
24:
25:   #define QS_INIT(arg_)        QS_init(arg_)
26:   #define QS_EXIT()            QS_exit()
27:
28:   #define QS_FILTER_ON(rec_)    QS_filterOn(rec_)
29:   #define QS_FILTER_OFF(rec_)   QS_filterOff(rec_)
30:
31:   /* QS record begin/end macros for application-specific records */
32:   #define QS_BEGIN_NOLOCK(rec_, obj_)          . . .
33:   #define QS_END_NOLOCK()                      . . .
34:   #define QS_BEGIN(rec_, obj_)                 . . .
35:   #define QS_END()                             . . .
36:
37:   /* QS record begin/end macros for pre-defined internal records */
38:   #define QS_BEGIN_NOLOCK_(rec_, objFilter_, obj_) . . .
39:   #define QS_END_NOLOCK_()                        . . .
40:   #define QS_BEGIN_(rec_, objFilter_, obj_)       . . .
41:   #define QS_END_()                               . . .
42:   . . .                                           /* QS data elements for pre-defined internal records */
43:
44:   /* QS data elements for application-specific records */
45:   #define QS_I8(width_, data_) . . .
46:   #define QS_U8(width_, data_) . . .
47:   #define QS_I16(width_, data_) . . .
48:   #define QS_U16(width_, data_) . . .
49:   #define QS_I32(width_, data_) . . .
50:   #define QS_U32(width_, data_) . . .
51:   #define QS_F32(width_, data_) . . .
52:   #define QS_F64(width_, data_) . . .
53:   #define QS_STR(str_) . . .
54:   #define QS_MEM(mem_, size_) . . .

```

```

55:
56:     #define QS_SIG(sig_, obj_)      . . .
57:     #define QS_OBJ(obj_)           . . .
58:     #define QS_FUN(fun_)           . . .
59:
60:     #define QS_SIG_DICTIONARY(sig_, obj_) . . .
61:     #define QS_OBJ_DICTIONARY(obj_) . . .
62:     #define QS_FUN_DICTIONARY(fun_) . . .
63:
64: #else                                     /* Q_SPY undefined--QS tracing DISABLED */
65:
66:     #define QS_INIT(arg_)           ((uint8_t)1)
67:     #define QS_EXIT()               ((void)0)
68:     #define QS_DUMP()               ((void)0)
69:     #define QS_FILTER_ON(rec_)      ((void)0)
70:     #define QS_FILTER_OFF(rec_)     ((void)0)
71:     #define QS_FILTER_SM_OBJ(obj_)  ((void)0)
72:     #define QS_FILTER_AO_OBJ(obj_)  ((void)0)
73:     #define QS_FILTER_MP_OBJ(obj_)  ((void)0)
74:     #define QS_FILTER_EQ_OBJ(obj_)  ((void)0)
75:     #define QS_FILTER_TE_OBJ(obj_)  ((void)0)
76:
77:     #define QS_BEGIN(rec_)          if (0) { ((void)0)
78:     #define QS_END()                } else ((void)0)
79:     #define QS_BEGIN_NOLOCK(rec_)   QS_BEGIN(rec_)
80:     #define QS_END_NOLOCK()         QS_END()
81:
82:     #define QS_BEGIN(rec_, obj_)    if (0) { ((void)0)
83:     #define QS_END()                } else ((void)0)
84:     #define QS_BEGIN_NOLOCK(rec_, obj_) QS_BEGIN(rec_, obj_)
85:     #define QS_END_NOLOCK()         QS_END()
86:
87:     #define QS_I8(width_, data_)    ((void)0)
88:     #define QS_U8(width_, data_)    ((void)0)
89:     #define QS_I16(width_, data_)   ((void)0)
90:     #define QS_U16(width_, data_)   ((void)0)
91:     #define QS_I32(width_, data_)   ((void)0)
92:     #define QS_U32(width_, data_)   ((void)0)
93:     #define QS_F32(width_, data_)   ((void)0)
94:     #define QS_F64(width_, data_)   ((void)0)
95:     #define QS_STR(str_)            ((void)0)
96:     #define QS_MEM(mem_, size_)     ((void)0)
97:     #define QS_SIG(sig_, obj_)      ((void)0)
98:     #define QS_OBJ(obj_)            ((void)0)
99:     #define QS_FUN(fun_)            ((void)0)
100:
101:     #define QS_SIG_DICTIONARY(sig_, obj_) ((void)0)
102:     #define QS_OBJ_DICTIONARY(obj_) ((void)0)
103:     #define QS_FUN_DICTIONARY(fun_) ((void)0)
104:
105: #endif                                     /* Q_SPY */

```

Listing 10 Platform-independent QS header file <qpc_3>\include\qs.h

Listing 10 shows the general structure of the qs.h header file. All QS facilities are provided in form of preprocessor macros. Depending on the global macro **Q_SPY**, the QS facilities are either defined to provide actual QS services (lines 2-62), or are “dummied-out” to prevent any code generation (lines 66-103) when the global macro **Q_SPY** is not defined. That way, the QS instrumentation can be left in the code at all times, but becomes active only when the code is compiled with the macro **Q_SPY** defined. You typically define the macro **Q_SPY** externally through the **-D** compiler option.

3.3 General Structure of QS Trace Records

As all software tracing systems, QS logs the tracing data in discrete chunks called QS trace records. These trace records have the following general structure:

```
QS_BEGINxxx(record_type);      /* trace record begin */
. . .                          /* QS data */
. . .                          /* QS data */
QS_ENDxxx();                   /* trace record end */
```

Listing 11 General structure of a QS record

Each trace record always begins with one variant of the macro `QS_BEGIN_xxx()` and ends with the matching macro `QS_END_xxx()`. Sandwiched between these two macros are the data-generating macros that actually insert individual data elements comprising the trace record into the QS trace buffer. QS provides four variants of the begin/end macro pairs for different purposes (Listing 10, lines 36-44).

The first two variants (Listing 10, lines 32-35) are for creating application-specific QS records (Section 3.6). The `QS_BEGIN()/QS_END()` pair locks interrupts at the beginning of the record and unlocks at the end. The pair `QS_BEGIN_NOLOCK()/QS_END_NOLOCK()` is for application-specific records without entering the QS critical and should be used only within already established critical sections.

The third and fourth variants of the begin/end-record QS macros (Listing 10, lines 38-41) are for internal use within QP components to generate the pre-defined QS records. Such pre-defined records are generated with `QS_BEGIN_()/QS_END_()` or `QS_BEGIN_NOLOCK_()/QS_END_NOLOCK_()` macro pairs, depending if QS critical section must be entered or not.

3.4 Dictionary QS Trace Records

By the time you compile and load your application image to the target, the symbolic information about the object names, function names, and signal names is stripped from the code. Therefore, if you want to have the symbolic information available to the QSpy host-resident component, you need to supply it somehow to the software tracing system.

QS provides special trace records designed expressly for including the symbolic information about the target code in the trace itself. The dictionary records included in the trace for the QSpy host application (see Section **Error! Reference source not found.**) are very much like the symbolic information included in the object files for the traditional single-step debugger.

The dictionary trace records are not absolutely required to generate the trace, in the same way as the symbolic information in the object files is not absolutely required to debug code. However, in both cases, the availability of the symbolic information greatly improves the productivity in working with the software trace or the debugger.

QS supports three types of dictionary trace records: object dictionary, function dictionary, and signal dictionary. The following sections cover these types in detail.

NOTE: As all QS trace records, the dictionary trace records are generated in a critical section of code, that is, interrupts are locked for the time the data is inserted into the QS trace buffer. Additionally, after unlocking interrupts, the callback function `QS_flush()` is invoked at the end of each dictionary record. This callback function typically busy-waits until all data is sent out to the host, which might take considerable time. For that reason, dictionary entries should be generated only during the system initialization, when the real-time constraints do not yet apply.

3.4.1 Object Dictionaries

Object dictionaries are generated with the macro `QS_OBJ_DICTIONARY()` that associates the address of the object in memory with its symbolic name. Listing 5, Listing 7, and Listing 8 provide examples of how you use this macro.

The `QS_OBJ_DICTIONARY()` macro takes only one argument: the address of the object, and uses internally the "strigization" preprocessor operator to convert the provided argument to a string. Therefore, you should invoke the `QS_OBJ_DICTIONARY()` macro with meaningfully named persistent objects, such as `&l_table`, or `&l_philo[3]`, and not generic pointers, such as "me", because the latter will not help you much to recognize the object name in the trace.

Object Type	Example(s)	QS Records
State machine	<code>QS_OBJ_DICTIONARY(&l_qhsmTst);</code> (Listing 3 line 57)	<code>QS_QEP_STATE_EMPTY,</code> <code>QS_QEP_STATE_ENTRY,</code> <code>QS_QEP_STATE_EXIT,</code> <code>QS_QEP_STATE_INIT,</code> <code>QS_QEP_INIT_TRAN,</code> <code>QS_QEP_INTERN_TRAN,</code> <code>QS_QEP_TRAN,</code> <code>QS_QEP_IGNORED</code>
Active object	<code>QS_OBJ_DICTIONARY(&l_philo[3]);</code> (Listing 8 line 19)	<code>QS_QF_ACTIVE_ADD,</code> <code>QS_QF_ACTIVE_REMOVE,</code> <code>QS_QF_ACTIVE_SUBSCRIBE,</code> <code>QS_QF_ACTIVE_UNSUBSCRIBE,</code> <code>QS_QF_ACTIVE_POST_FIFO,</code> <code>QS_QF_ACTIVE_POST_LIFO,</code> <code>QS_QF_ACTIVE_GET,</code> <code>QS_QF_ACTIVE_GET_LAST</code>
Memory pool ⁽¹⁾	<code>QS_OBJ_DICTIONARY(l_regPoolSto);</code> (Listing 5 line 28)	<code>QS_QF_MPOOL_INIT,</code> <code>QS_QF_MPOOL_GET</code> <code>QS_QF_MPOOL_PUT,</code>
Event queue ⁽²⁾	<code>QS_OBJ_DICTIONARY(l_philQueueSto[3]);</code> (Listing 5 line 34)	<code>QS_QF_EQUEUE_INIT,</code> <code>QS_QF_EQUEUE_POST_FIFO,</code> <code>QS_QF_EQUEUE_POST_LIFO,</code> <code>QS_QF_EQUEUE_GET,</code> <code>QS_QF_EQUEUE_GET_LAST</code>
Time event	<code>QS_OBJ_DICTIONARY(&l_philo[3].timeEvt_);</code> (Listing 8 line 20)	<code>QS_QF_TICK,</code> <code>QS_QF_TIMEEVT_ARM,</code> <code>QS_QF_TIMEEVT_AUTO_DISARM,</code> <code>QS_QF_TIMEEVT_DISARM_ATTEMPT,</code> <code>QS_QF_TIMEEVT_DISARM,</code> <code>QS_QF_TIMEEVT_REARM,</code> <code>QS_QF_TIMEEVT_POST,</code> <code>QS_QF_TIMEEVT_PUBLISH</code>

Table 2 Object dictionaries required for the pre-defined QS records. (1) Memory pool is referenced by the memory buffer managed by the pool. (2) Event queue is referenced by the ring buffer managed by the queue.

Table 2 enlists object dictionaries you need to provide in order to provide the symbolic information used by the specific pre-defined QS records. Please note that QS identifies memory pools by the memory buffer managed by the memory pool, because the actual memory pool objects are buried inside the framework and are not accessible to the application developer. Also, event queues are identified by the ring buffer managed by the queue, not by the queue object itself.

The object dictionary records are closely related to the QS local filters (see Section 3.5.2). Both facilities consistently use the same conventions. For example, local filter for a specific memory pool is

set up by providing the memory buffer managed by the memory pool to the `QS_FILTER_MP_OBJ()` macro. Similarly, a local filter for a specific event queue is set up by specifying the ring buffer to the `QS_FILTER_EQ_OBJ()` macro.

3.4.2 Function Dictionaries

Function dictionaries are generated with the macro `QS_FUN_DICTIONARY()` that associates the address of the function in memory with its symbolic name. Listing 5, Listing 7, and Listing 8 provide examples of how you use this macro. The main purpose of the function dictionaries is to provide symbolic names for state handler functions.

The `QS_FUN_DICTIONARY()` macro takes only one argument: the address of the function, and uses internally the "strigization" preprocessor operator to convert the provided argument to a string.

3.4.3 Signal Dictionaries

Signal dictionaries are generated with the macro `QS_SIG_DICTIONARY()` that associates the numerical value of the event signal and the state machine object to the symbolic name of the signal.

The reason for using both the signal value and the state machine object, rather than just the signal value, is that a signal value alone is not sufficient to uniquely identify the symbolic signal. Only the globally published signals are required to be system-wide unique. Other signals, used only locally, can have completely different meaning for different state machines in the system.

The `QS_SIG_DICTIONARY()` macro takes two arguments: the numerical value of the signal and the address of the state machine object. The macro uses internally the "strigization" preprocessor operator to convert the provided signal argument to a string. The state machine object is not converted to a string, so the actual variable name you use is irrelevant (see line 32 in Listing 8).

Listing 5 and Listing 8 provide examples of how you use the `QS_SIG_DICTIONARY()` macro. Listing 5 shows how to specify globally published signals that are associated with multiple state machines. In this case, you specify `NULL` as the state machine object. In contrast, line 31 of Listing 8 shows a dictionary entry for the local signal `TIMEOUT_SIG`. This signal is associated only with the Philosopher state machines.

3.5 QS Filters

One of the main roles of the begin-record macros is to implement the filtering of QS records before they reach the trace buffer. QS provides two complementary levels of filtering: the global on/off filter and local filters (see Figure 9).

3.5.1 Global On/Off Filter

The global on/off filter is based on the record IDs. The `qs.h` header file provides the enumeration of all the pre-defined internal trace records IDs that are already instrumented into the QP components (Listing 10, lines 4-9). The enumeration of the pre-defined records ends with the `QS_USER` value, which is the first numerical value available for application-specific trace records. Section 3.6 discusses the application specific trace records in detail.

The global on/off filter is efficiently implemented by means of an array of bitmasks `QS_glbFilter_[]` (Listing 10, line 11), where each bit represents one trace record. Currently the `QS_glbFilter_[]` array contains 32 bytes for the total of 32×8 bits for 256 different trace records. A little more than a quarter of these records are already taken by the pre-defined QP trace records. The remaining three quarters are available for application-specific use.

The beginning of the macro `QS_BEGIN()` shows how the global on/off filter is implemented:

```
#define QS_BEGIN(rec_, obj_) \  
    if (((QS_glbFilter_[(uint8_t)(rec_) >> 3U] \  
        & (1U << ((uint8_t)(rec_) & 7U))) != 0) \  
        . . .
```

The global on/off filter works by checking the state of the bit corresponding to the given trace record `rec_`. This check is accomplished by the familiar expression `(bimask & bit) != 0`, where the bitmask is `QS_glbFilter_[(uint8_t)(rec_) >> 3U]`, and the bit is `(1U << ((uint8_t)(rec_) & 7U))`. Please note that for any constant value of the argument `rec_`, both the bitmask and the bit are compile-time constants as well, so the global filter check for a record ID of 46, say, costs as much as the expression: `(QS_glbFilter_[5] & 0x40) != 0`.

NOTE: the global filter is specifically implemented to use byte-size computations only to be efficient even on 8-bit machines.

QS provides a simple interface for setting and clearing individual bits in the `QS_glbFilter_[]` bitmask array. The macro `QS_FILTER_ON(rec_)` turns on the bit corresponding to record “`rec_`”. Conversely, the macro `QS_FILTER_OFF(rec_)` turns off the bit corresponding to record “`rec_`”. In both cases, the special constant `QS_ALL_RECORDS` affects all records. Specifically, `QS_FILTER_ON(-QS_ALL_RECORDS)` turns on all records, and `QS_FILTER_OFF(QS_ALL_RECORDS)` turns off all records. Examples of these macros are provided in Listing 3 and Listing 5.

Just after QS initialization, the global on/off filter is set to OFF for all records IDs. You need to explicitly turn the filter ON for some records to enable the tracing.

NOTE: Globally disabling all records through `QS_FILTER_OFF(QS_ALL_RECORDS)` is a useful way of implementing a software tracing **trigger**. You can use this trigger to rapidly stop the tracing after an interesting event, to prevent new trace data from overwriting interesting data in case the data uplink to the host cannot keep up with the production rate of new trace data.

3.5.2 Local Filters

The local filters allow generation of trace records only for specified objects. For example, you might setup a local filter to log only activity of a given state machine object. Independently, you might setup another local filter to log only activity of a given memory pool.

The following table summarizes all specified local filters and the pre-defined QS records controlled by these filters:

Local Filter	Object Type	Example	QS Records
QS_FILTER_SM_OBJ()	State machine	QS_FILTER_SM_OBJ(&l_qhsmTst);	QS_QEP_STATE_EMPTY, QS_QEP_STATE_ENTRY, QS_QEP_STATE_EXIT, QS_QEP_STATE_INIT, QS_QEP_INIT_TRAN, QS_QEP_INTERN_TRAN, QS_QEP_TRAN, QS_QEP_IGNORED
QS_FILTER_AO_OBJ()	Active object	QS_FILTER_AO_OBJ(&l_philo[3]);	QS_QF_ACTIVE_ADD, QS_QF_ACTIVE_REMOVE, QS_QF_ACTIVE_SUBSCRIBE, QS_QF_ACTIVE_UNSUBSCRIBE, QS_QF_ACTIVE_POST_FIFO, QS_QF_ACTIVE_POST_LIFO, QS_QF_ACTIVE_GET, QS_QF_ACTIVE_GET_LAST
QS_FILTER_MP_OBJ() ⁽¹⁾	Memory pool	QS_FILTER_MP_OBJ(l_regPoolSto);	QS_QF_MPOOL_INIT, QS_QF_MPOOL_GET, QS_QF_MPOOL_PUT,
QS_FILTER_EQ_OBJ() ⁽²⁾	Event queue	QS_FILTER_EQ_OBJ(l_philQueueSto[3]);	QS_QF_EQUEUE_INIT, QS_QF_EQUEUE_POST_FIFO, QS_QF_EQUEUE_POST_LIFO, QS_QF_EQUEUE_GET, QS_QF_EQUEUE_GET_LAST
QS_FILTER_TE_OBJ()	Time event	QS_FILTER_TE_OBJ(&l_philo[3].timeEvt_);	QS_QF_TICK, QS_QF_TIMEEVT_ARM, QS_QF_TIMEEVT_AUTO_DISARM, QS_QF_TIMEEVT_DISARM_ATTEMPT, QS_QF_TIMEEVT_DISARM, QS_QF_TIMEEVT_REARM, QS_QF_TIMEEVT_POST, QS_QF_TIMEEVT_PUBLISH
QS_FILTER_AP_OBJ()	Generic application object	QS_FILTER_AP_OBJ(&myAppObject);	Application-specific records Starting with QS_USER ...

Table 3 Local filter summary. (1) Memory pool is referenced by the memory buffer managed by the pool. (2) Event queue is referenced by the ring buffer managed by the queue.

The first column of Table 3 enlists the QS macros you need to use in order to set/clear the local filters. For example, you specify the state machine local filter by invoking:

```
QS_FILTER_SM_OBJ(myStateMachinePointer);
```

where myStateMachinePointer is the pointer to the state machine object you want to trace.

You deactivate any local filter by passing the NULL pointer to the appropriate QS macro. For example, to open-up the local filter for all state machine objects, you write the following code:

```
QS_FILTER_SM_OBJ(0);
```

Just after QS initialization all local filters is set to NULL, meaning that the local filters are open for all objects.

The highlighted code in the QS_BEGIN() macro definition shows the actual implementation of the local filter for the application-specific objects:

```
#define QS_BEGIN(rec_, obj_) \
    if (((QS_glbFilter_[(uint8_t)(rec_) >> 3U] \
        & (1U << ((uint8_t)(rec_) & 7U))) != 0) \
        && ((QS_apObj_ == (void *)0) || (QS_apObj_ == (obj_)))) \
    { \
        . . .
```

The QS local filters are closely related to the object dictionary records (see Section 3.4.1) and both facilities consistently use the same conventions.

3.6 Application-Specific QS Trace Records

As a QS application developer, you need to be concerned only with the application-specific trace records. The example of a very simple application-specific record is provided in lines 99-101 of Listing 3. The following Listing 12 shows an example of an application-specific QS record with some more data elements:

```
QS_BEGIN(MY_QS_RECORD, myObjectPointer);           /* trace record begin */
  QS_STR("Hello");                                /* string data element */
  QS_U8(3, n);                                     /* uint8_t data, 3-decimal digits format */
  . . .                                           /* QS data */
  QS_MEM(buf, sizeof(buf));                       /* memory block of a given size */
QS_END();                                           /* trace record end */
```

Listing 12 Example of an application-specific trace record

In most cases, the application-specific records are enclosed with the `QS_BEGIN()/QS_END()` pair of macros. This pair of macros locks interrupts at the beginning and unlocks at the end of each record (see Section 3.7). Occasionally, you would want to generate trace data from within already established critical sections or ISRs. In such rare occasions, you would use the macros `QS_BEGIN_NOLOCK()/QS_END_NOLOCK()` to avoid nesting of critical sections.

The record-begin macro `QS_BEGIN()` takes two arguments. The first argument (e.g., `MY_QS_RECORD`) is the enumerated record type, which is used in the global on/off filter (Section 3.5.1) and is part of the each record header. The application-specific record types must start with the value `QS_USER` in order to avoid overlap with the pre-defined QS records already instrumented into the QP components.

The second argument (e.g., `myObjectPointer`) is used for the local filter, which allows you to selectively log only specific application-level objects. For example, the application record beginning in line 99 of Listing 3 pertains to the state machine object "me". This means that you have an option to use the application-specific local filter `QS_FILTER_AP_OBJ(&l_qhsmTst)` (see Section 3.5.2) to selectively trace only the `l_qhsmTst` object. You can always disable the application-specific local filter by using `QS_apobj_` as the second argument of the macro `QS_BEGIN()`, or `QS_BEGIN_NOLOCK()`.

Sandwiched between the `QS_BEGIN()/QS_END()` macros are data elements that you want to store in the trace record. The macros for generating the data elements are shown in lines 45-54 of Listing 10. The supported data elements include: signed and unsigned integers of 8-bit, 16-bit and 32-bit size, floating point numbers of 32-bit and 64-bit size, zero-terminated strings, and variable-size memory blocks. Special macros are also provided for tracing configuration-dependent elements, such as event signals, object pointers, and function pointers. For these configurable or platform-specific data elements, QS logs only the minimal number of bytes required on the given platform.

As illustrated in Listing 12, you can place many data elements of any kind in any order inside an application-specific record. The data protocol used to store the data is such that the data-type information is stored along the data itself, so that any application-specific record can always be parsed without a-priori knowledge of its content. The only limitation is that a complete record must fit in the QS trace buffer. Of course, you should avoid such big trace records anyway, because QS records are always placed in the buffer within a critical section of code. Also, you might want to conserve the buffer space.

The numerical data elements allow you to specify the formatting of the value by the host-resident QSpy component. For example, the data `QS_U8(2, n)` will cause the value 'n' to be formatted on 2 decimal digits. The maximum allowed format width is 15 decimal digits.

3.7 QS Critical Section

QS must software tracing of any code, including concurrent tasks and interrupts. In order to protect the internal ring buffer (see Figure 9) from data corruption, QS needs to access the buffer exclusively. For such exclusive access, QS uses the same mechanism as the rest of the Quantum Platform, that is, locking interrupts on entry to the **critical section** of code and unlock interrupts again on exit.

NOTE: When QS is combined with QF or QK, the QS critical section implementation must be identical to that used in the QF. However, QS can also be used with just the QEP component, or even completely standalone. In these cases, QS must provide it's own, independent critical section mechanism.

For compatibility with the rest of the Quantum Platform, QS uses the critical section mechanism as described in Section 5.2 of the "[QF/C Programmer's Manual](#)". The following three QS macros define the platform-specific interrupt locking/unlocking policy of QS:

```
#define QS_INT_KEY_TYPE      . . .  
#define QS_INT_LOCK(key_)   . . .  
#define QS_INT_UNLOCK(key_) . . .
```

Listing 13 QS macros for interrupt locking and unlocking

These QS macros are exactly analogous to the QF macros `QF_INT_KEY_TYPE`, `QF_INT_LOCK()`, and `QF_INT_UNLOCK()`, respectively. Please refer to the aforementioned Section 5.2 of the "[QF/C Programmer's Manual](#)" for more details.

3.8 QS Data Protocol

The data transmission protocol used in QS to transmit trace data from the target to the host is one of its greatest strengths. The protocol is very lightweight, but has many elements of the High Level Data Link Control (HDLC) protocol defined by the International Standards Organization (ISO).

The QS protocol has been specifically designed to simplify the data management overhead in the target, yet to allow detection of any data dropouts due to the trace buffer overruns. The protocol has not only provisions for detecting gaps in the data and other errors, but allows for instantaneous resynchronization after any error to minimize data loss.

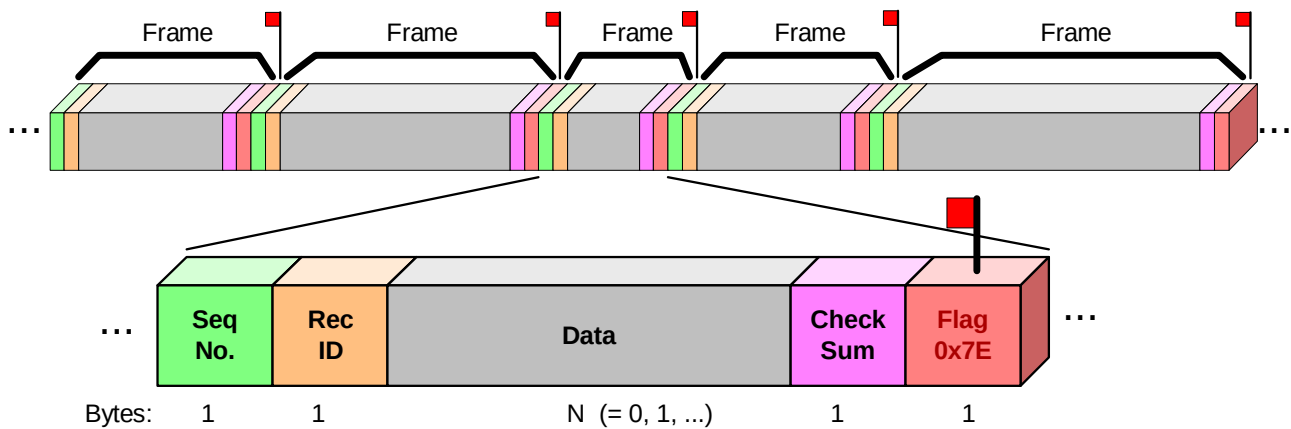


Figure 10 QS transmission protocol

The QS protocol transmits each trace record in an HDLC-like frame. The upper part of Figure 10 shows the serial data stream transmitted from the target containing frames of different lengths. The bottom part of Figure 10 shows the details of a single frame:

1. Each frame starts with the Frame Sequence Number byte. The target QS component increments the Frame Sequence Number for every frame inserted into the circular buffer. The Sequence Number naturally rolls-over from 255 to 0. The Frame Sequence Number allows the QSpy host component to detect any data discontinuities.
2. Following the Frame Sequence Number is the Record ID byte.
3. Following the Record ID is zero or more Data bytes.
4. Following the data is the Checksum. The Checksum is computed over the Frame Sequence Number, the Record ID, and all the Data bytes. The next section gives the detailed checksum computation formula.
5. Following the Checksum is the HDLC Flag, which delimits the frame. The HDLC flag is the 01111110 binary string, that is, 0x7E hexadecimal. Please note that the QS protocol uses only one HDLC Flag at the end of each frame. In other words, only one Flag is inserted between frames.

The QS target component performs the HDLC-like framing described above at the time the bytes are inserted into the circular trace buffer. This means that the data in the buffer is already cleanly divided into frames and can be transmitted in any chunks, typically not aligned with the frame boundaries.

3.8.1 Transparency

One of the most important characteristics of HDLC-type protocols is establishing very easily identifiable frames in the serial data stream. The receiver end of such protocol can instantaneously synchronize to the frame boundary by simply finding the Flag byte. This is because the special Flag byte can never occur within the content of a frame. To avoid confusing unintentional Flag bytes that can naturally occur in the data stream with an intentionally sent Flag, HDLC uses a technique known as "byte stuffing" or "escaping" to make the Flag bytes *transparent* during the transmission. Whenever the transmitter encounters a Flag byte in the data, it inserts a *two*-byte escape sequence to the output stream. The first byte is the Escape byte, defined as binary 01111101 (hexadecimal 0x7D). The second byte is the original byte XORed with 0x20.

Of course, now the Escape byte itself must also be transparent to avoid interpreting an unintentional Escape byte as the two-byte escape sequence. The procedure of escaping the Escape byte is identical to that of escaping the Flag byte.

The transparency of the Flag and Escape bytes complicates slightly the computation of the checksum. The transmitter computes the checksum over the Frame Sequence Number, the Record ID, and all Data bytes *before* performing any "escaping". The receiver must apply the exact reversed procedure of performing the "byte un-stuffing" before computing the checksum.

An example may make this more clear. Suppose that the following trace record needs to be inserted to the trace buffer (the transparent bytes are shown in bold):

```
Record ID = 0x7D, Record Data = 0x7D 0x08 0x01
```

Assuming that the current Frame Sequence Number is, say **0x7E**, the Checksum will be computed over the following bytes:

```
Checksum == (uint8_t)(~(0x7E + 0x7D + 0x7D + 0x08 + 0x01)) == 0x7E
```

and the actual frame inserted into the QS trace buffer will be as follows:

```
0x7D 0x5E 0x7D 0x5D 0x7D 0x5D 0x08 0x01 0x7D 0x5E 0x7E
```

Obviously, this is a degenerated example, where the Frame Sequence Number, the Record ID, a Data byte, and the Checksum itself turned out to be the transparent bytes. Typical overhead of transparency with real trace data is one escape sequence per several trace records.

3.8.2 Endianness

In addition to the HDLC-like framing, the QS transmission protocol specifies the endianness of the data to be **little-endian**. All multi-byte data elements, such as 16- and 32-bit integers, pointers, and floating point numbers are inserted into the QS trace buffer in the little-endian byte order (least-significant byte first). The QS data inserting macros (see Listing 10 lines 45-52) place the data in the trace buffer in a platform-neutral manner, meaning that the data is inserted into the buffer in the little-endian order regardless of the endianness of the CPU. Also, the data-inserting macros copy the data to the buffer one byte at a time, thus avoiding any potential address misalignment problems. Many embedded CPUs, such as ARM, require certain alignment of 16- and 32-bit quantities.

3.9 QS Trace Buffer

As described in the previous section, the QS target component performs the HDLC-like framing at the time the bytes are inserted into the QS trace buffer. This means that only HDLC-like frames are placed in the buffer, which is the pivotal point in the design of the whole QS target component and has two important consequences.

First, the use of formatted data in the trace buffer allows to decouple the data insertion into the trace buffer from the data removal out of the trace buffer. You can simply remove the data in whichever chunks you like, without any consideration for frame boundaries and without any additional overhead of adding the protocol formatting. You can employ just about any available physical communication link available on the target for the efficient data transfer.

Second, the use of the formatted data in the buffer enables the “last is best” tracing policy. The QS transmission protocol maintains both the Frame Sequence Number and the Checksum over each trace record, which means that any data corruption caused by overrunning the old data with the new data can be reliably detected. Therefore, the new trace data is simply inserted into the circular trace buffer, regardless if it perhaps overwrites the old data that hasn't been sent out yet, or is in the process of being sent. The burden of detection any data corruption is placed on the QSpy host component. When you start missing the frames (which the host component easily detects by discontinuities in the Frame Sequence Number), you have several options. You can apply some additional filtering, increase the size of the buffer, or change the data transfer policy¹.

3.9.1 Byte-Oriented Interface QS_getByte()

The lack of any constraints on removing the data from the trace buffer means that you can remove one byte at a time at arbitrary time instances. QS provides the function `QS_getByte()` for such byte-oriented interface. Listing 6 shows an example how to use this function.

The signature of `QS_getByte()` is as follows:

```
uint16_t QS_getByte(void);
```

The function returns the byte in the least-significant 8-bits of the 16-bit return value if the byte is available. If the trace buffer has no more data, the function returns `QS_EOD` (End-Of-Data), which is defined in `qs.h` as `((uint16_t)0xFFFF)`.

NOTE: The function `QS_getByte()` does not lock interrupts internally and is not reentrant. In particular, you should always design your software such that `QS_getByte()` must return before being called again.

NOTE: An application should use consistently either the byte-oriented interface `QS_getByte()` or the block-oriented interface `QS_getBlock()` (see the next section), but never both at the same time.

3.9.2 Block-Oriented Interface QS_getBlock()

QS also provides an alternative block-oriented interface for obtaining the data a contiguous block at time. QS provides the function `QS_getBlock()` for such block-oriented interface. Such block-

¹ The data transfer policy shown in the QDPP-SPY example is to use only the idle CPU cycles to transfer the trace data. At the cost of increased invasiveness, you might decide to perform data transfer more often.

oriented interface is very useful for starting DMA-type transfers. Listing 3 shows an example how to use this function.

The signature of `QS_getBlock()` is as follows:

```
uint8_t const *QS_getBlock(uint16_t *pnBytes);
```

If any bytes are available at the time of the call, the function returns the pointer to the beginning of the contiguous data block, and writes the number of bytes in the block to the location pointed to by `pnBytes`. The argument `pnBytes` is also used as input to limit the maximum size of the data block that the caller can accept.

If no bytes are available in the QS buffer when the function is called, the function returns a NULL pointer and sets the value pointed to by `pnBytes` to zero.

After the `QS_getBlock()` returns a memory block to the caller, the caller must transfer all the bytes in the returned block before calling `QS_getBlock()` again. You should not assume that after `QS_getBlock()` returns a block, the QS trace buffer is necessarily empty. Sometimes, the block is at the end of the buffer and you need to call `QS_getBlock()` again to obtain the rest of the data that "wrapped around" to the beginning of the QS data buffer.

NOTE: The function `QS_getBlock()` does not lock interrupts internally and is not reentrant. In particular, you should always design your software such that `QS_getBlock()` must return before being called again.

As an exercise, you might want to replace the byte-oriented interface in the QDPP-SPY application with the block-oriented one. The following code illustrates the changes that you need to do in the file `<qpc_3>\examples\80x86\qk\tcpp101\1\qdpp-spy\bsp.c`:

```
#ifdef Q_SPY
#define QS_DUMP() \
    if ((inportb(l_base + 5) & (1 << 5)) != 0) { \
        uint16_t nBytes = UART_16550_TXFIFO_DEPTH; \
        uint8_t const *block = QS_getBlock(&nBytes); \
        while (nBytes -- != 0) { \
            outportb(l_base + 0, *block++); \
        } \
    } else ((void)0)
/*.....*/
void QS_flush(void) {
    uint16_t nBytes = UART_16550_TXFIFO_DEPTH;
    uint8_t const *block;
    while ((block = QS_getBlock(&nBytes)) != (uint8_t *)0) {
        while ((inportb(l_base + 5) & (1 << 5)) == 0) { /*TX FIFO not empty? */
        }
        while (nBytes-- != 0) {
            outportb(l_base + 0, *block++); \
        }
        nBytes = UART_16550_TXFIFO_DEPTH;
    }
}
#endif /* Q_SPY */
```

3.10 Configuring and Porting QS

When you use QS in your application, you are responsible for providing the application code and the adapt of QS to the CPU, compiler, and CPU/compiler options of your choice. Such adaptation is called a QS port. The standard QS distribution contains the platform-independent code, as well as the platform-specific code for the 80x86 example for DOS and QF/QK. If you intend to use QS on a different processor, you need to obtain a QS port to that CPU (look at www.quantum-leaps.com/downloads/ports.htm), or you need to port QS yourself, which is really simple. Please note that if you intend to use QS with QF, you must use the same interrupt locking policy in both components (see Section 3.7).

Perhaps the most important aspect to understand for porting QS is the Quantum Platform code organization. To this end, the Quantum Leaps Application Note "[Quantum Platform Directory Structure](#)" describes in detail the directories, files, and the general structure of the platform-specific code.

The QS platform-specific code consists only of `qs_port.h` and QS callbacks defined typically in the Board Support Package (`bsp.c`) of your application. The header file `qs_port.h` is placed in the include directory for the port `<qpc_3>\ports\<cpu>\<os>\<compiler>\<options>`.

3.10.1 The `qs_port.h` File

The QS port file `<qpc_3>\ports\<cpu>\<os>\<compiler>\<options>\qs_port.h` specifies the following macros:

Macro Name	Example	Comments
<code>QS_TIME_SIZE</code>	4	The size (in bytes) of the QS timestamp representation (data type <code>QSTimeCtr</code>). Valid values: 1, 2, or 4; default 4.
<code>QS_OBJ_PTR_SIZE</code>	2	The size (in bytes) of the object pointer representation. Valid values: 1, 2, or 4; default 4.
<code>QS_FUN_PTR_SIZE</code>	4	The size (in bytes) of the function pointer representation. Valid values: 1, 2, or 4; default 4.
<code>QS_INT_KEY_TYPE</code>	<code>int</code>	If defined, this macro specifies the type of the interrupt lock key (see Section 3.7).
<code>QS_INT_LOCK(key_)</code>	<code>disable()</code>	Defines the mechanism of locking interrupts.
<code>QS_INT_UNLOCK(key_)</code>	<code>enable()</code>	Defines the mechanism of unlocking interrupts.

Table 4 Summary of `qs_port.h` macros.

After defining these macros for the given platform, the `qs_port.h` file must include the platform-independent QS header file `qs.h`.

3.10.2 The QS Callbacks in the BSP

Function Name	Comments
QS_init()	Initialization of QS invoked through the macro QS_INIT(). At a minimum, the function must configure the QS buffer by calling QS_initBuf(). Typically, you will also want to open/configure the QS output data link, such as a serial port, or a data file. The void* argument can be used to pass parameter(s) needed to configure the output data link. The function must return TRUE (1) if the QS initialization was successful, or FALSE (0) if it failed.
QS_exit()	Cleanup of QS invoked through the macro QS_EXIT(). The main purpose of this function is to close the QS output data link, if necessary.
QS_getTime()	Callback to add the time stamp to the QS record invoked from the macro QS_TIME(). Please note that some of the pre-defined QS records from QP do not output the time stamp. However, all application-specific records do output the time stamp. NOTE: QS_getTime() is called in a critical section and should not unlock interrupts.
QS_flush()	Flush the QS trace data to the host. This function is invoked at the end of each QS dictionary record, outside of critical section. QS_flush() typically busy-waits until all the data in the buffer is sent to the host. This is acceptable only in the initial transient.

Table 5 Summary of QS callback functions to implement at the application level (typically in the Board Support Package bsp.c).

4 QS Compliance with Standards

4.1 MISRATM Compliance

QS 3.2 source code is 98% compliant with the Motor Industry Software Reliability Association (MISRA) *Guidelines For The Use of The C Language in Vehicle Based Software* [MISRA 98]. These standards were created by MISRATM to improve the reliability and predictability of C programs in critical automotive systems. Members of the MISRATM consortium include Delco Electronics, Ford Motor Company, Jaguar Cars Ltd., Lotus Engineering, Lucas Electronics, Rolls-Royce, Rover Group Ltd., and other commercial and academic institutions dedicated to improving safety and reliability in automotive electronics. Full details of this standard can be obtained directly from the MISRATM web site, www.misra.org.uk.

The Application Note "QS MISRA Compliance Matrix" shows how QS complies with all 127 C Coding Rules. The document is available online from http://www.quantum-leaps.com/doc/AN_QEPC_MISRA.pdf.

4.2 PC-LintTM Compliance

QS 3.2 source code is "lint-free". The compliance was checked against PC-lintTM/FlexLintTM, which is perhaps the most advanced and widely used static analysis tool from Gimpel Software (<http://www.gimpel.com>). The QS distribution includes the <qpc_3>/qs/lint/ subdirectory, which contains the PC-lintTM configuration files used for static analysis of the QS source code.

The Application Note "QS PC-Lint Compliance Report" reports how QS complies with PC-lintTM and discusses the configuration used. The document is available online from http://www.quantum-leaps.com/doc/AN_QEP_C_PC-lint.pdf.

4.3 Coding Standard Compliance

QS 3.2 source code is compliant with the documented "Quantum Leaps C/C++ Coding Standard" available online from <http://www.quantum-leaps.com/resources/goodies.htm#CodingStandard>.

4.4 QS Reference Manual

The QS Reference Manual is available online in HTML format from http://www.quantum-leaps.com/-doxygen/qpc_3.1 and is also available in a compressed format for download from http://www.quantum-leaps.com/doxygen/qpc_3.1.zip.

The Reference Manual has been generated by Doxygen (<http://www.doxygen.org>), which is an open source documentation generation system for C, C++, Java, and other languages.

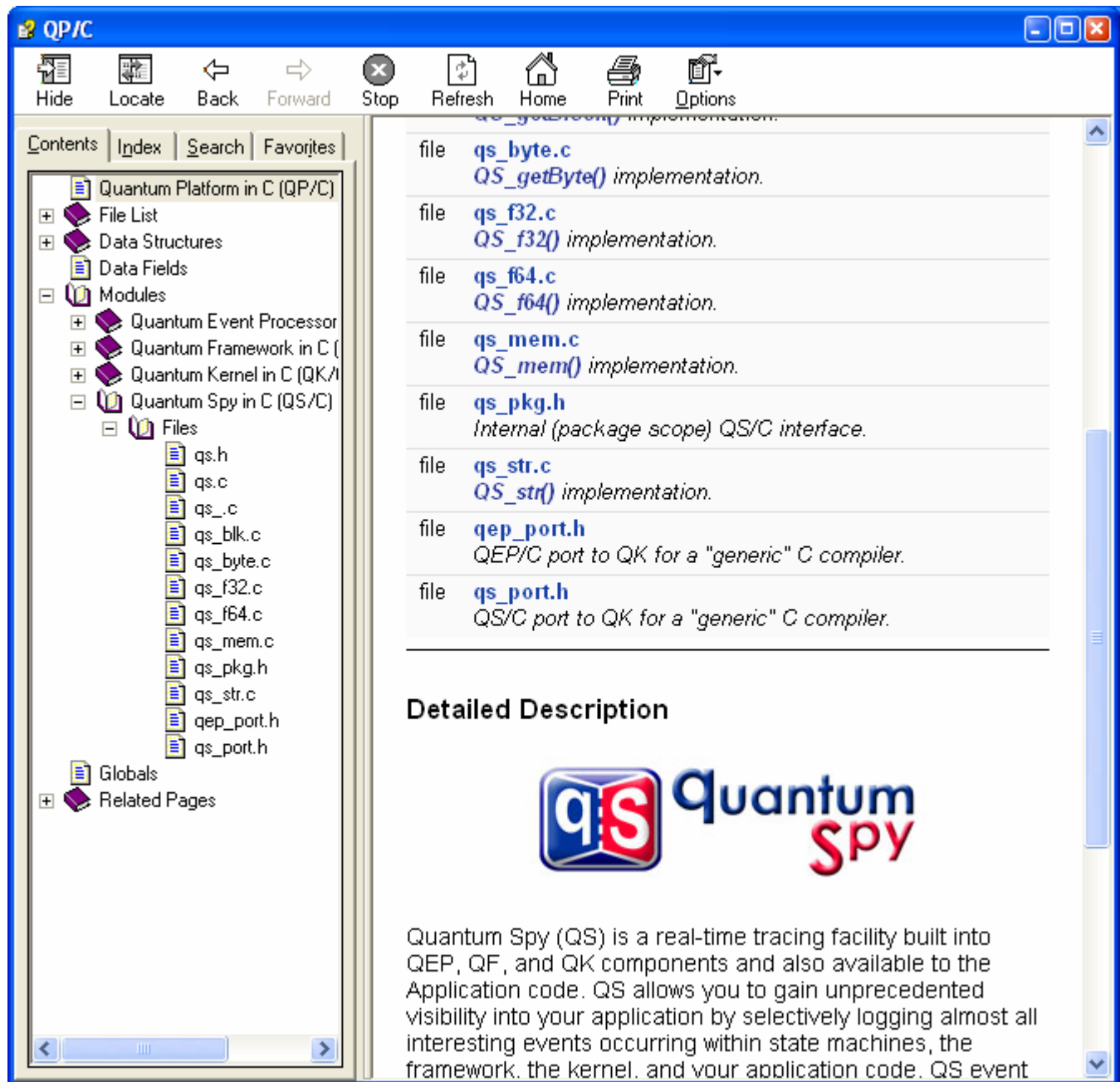


Figure 11 Screen shot of the QS/C Reference Manual.

5 Related Documents and References

Document	Location
"QSpy TM Host Application User's Manual", Quantum Leaps, LLC 2006	http://www.quantum-leaps.com/doc/-QSpy_Manual.pdf
"QL Licensing and Support", Quantum Leaps, LLC 2005	http://www.quantum-leaps.com/doc/-QP_Overview.pdf
"QP Directory Structure", Quantum Leaps, LLC 2005	http://www.quantum-leaps.com/doc/-AN_QP_Directory_Structure.pdf
"Quantum Platform Overview", Quantum Leaps, LLC 2005	http://www.quantum-leaps.com/doc/-QP_Overview.pdf
"QEP/C Programmer's Manual", Quantum Leaps, LLC, 2005	http://www.quantum-leaps.com/doc/-QEP_C_Manual.pdf
"QF/C Programmer's Manual", Quantum Leaps, LLC 2005	http://www.quantum-leaps.com/doc/-QF_C_Manual.pdf
"QK/C Programmer's Manual", Quantum Leaps, LLC 2005	http://www.quantum-leaps.com/doc/-QK_C_Manual.pdf
[C99] ISO/IEC 9899 C Approved Standards	http://www.open-std.org/jtc1/sc22/-open/n2794
[MISRA 98] Motor Industry Software Reliability Association (MISRA), MISRA Limited, MISRA-C:1998 Guidelines for the Use of the C Language in Vehicle Based Software, April 1998, ISBN 0-9524156-9-0	See at: http://www.misra.org.uk
[OMG 03] Unified Modeling Language: Superstructure,	document ptc/03-08-02.pdf, See also http://www.omg.org